



Module 1 — SQL Programming

Learn how to store, retrieve, and analyze structured data using MySQL — covering core SQL, Joins, Subqueries, Set Operations, and Views essential for AI & Data Science.

Author
Srinivas Dande



**Dandes
Academy**



Module 1 — SQL Programming

1. Introduction to Databases & SQL

- 1.1. What is a Database?
- 1.2. What is SQL?
- 1.3. Difference between SQL vs NoSQL
- 1.4. Tables, Rows, Columns
- 1.5. Types of Keys
- 1.6. Basic Database Terminology

2. DB Setup and SQL Fundamentals

- 2.1. Installing MySQL Server
- 2.2. Installing MySQL Workbench
- 2.3. Work with MySQL Workbench
- 2.4. Creating a New Database (Schema)
- 2.5. Creating a Table
- 2.6. Inserting Data
- 2.7. Viewing Data
- 2.8. Updating Data
- 2.9. Deleting Data
- 2.10. ALTER Command
- 2.11. Dropping a Table
- 2.12. Truncating a Table
- 2.13. SQL Command Types

3. Data Types and Constraints

- 3.1. SQL Data Types
- 3.2. SQL Constraints
- 3.3. NOT NULL Constraint
- 3.4. UNIQUE Constraint
- 3.5. PRIMARY KEY Constraint
- 3.6. FOREIGN KEY Constraint
- 3.7. DEFAULT Constraint
- 3.8. CHECK Constraint
- 3.9. AUTO INCREMENT



4. Filtering and Sorting

- 4.1. SELECT Statement
- 4.2. SQL Operators
- 4.3. Practice Lab — Filtering and Sorting
- 4.4. Common Query Examples
- 4.5. ORDER BY Clause

5. Aggregation and Grouping

- 5.1. What are Aggregate Functions?
- 5.2. Practice Lab — Aggregation and Grouping
- 5.3. GROUP BY Clause
- 5.4. HAVING Clause
- 5.5. Handling NULLs in Aggregates

6. Joins and Relationships

- 6.1. Table Relationships
- 6.2. What is a JOIN?
- 6.3. Types of Joins
- 6.4. Practice Lab – Joins
- 6.5. Inner Join (Equi Join)
- 6.6. Left Outer Join
- 6.7. Right Outer Join
- 6.8. Full Outer Join
- 6.9. JOINS with Extra Conditions
- 6.10. Self Join
- 6.11. Cross Join

7. Subqueries

- 7.1. What is a Subquery?
- 7.2. Rules of Subqueries
- 7.3. Types of Subqueries
- 7.4. Practice Lab - Subqueries
- 7.5. Creating New Tables from Queries



8. Set Operations

- 8.1. What are Set Operations?
- 8.2. Rules for Using Set Operators
- 8.3. Types of Set Operations
- 8.4. Practice Lab – Set Operations
- 8.5. ORDER BY with Set Operations

9. Views

- 9.1. Introduction to Views
- 9.2. Practice Lab – Views
- 9.3. Types of Views (by updatability)

10. SQL Built-In Functions

- 10.1. Arithmetic Functions
- 10.2. String Functions
- 10.3. Date & Time Functions
- 10.4. Conversion Functions
- 10.5. Aggregate Functions



1. Introduction to Databases & SQL

1.1. What is a Database?

1.2. What is SQL?

1.3. Difference between SQL vs NoSQL

1.4. Tables, Rows, Columns

1.5. Types of Keys

1.6. Basic Database Terminology



1.1. What is a Database?

- Database is an organized collection of data stored and accessed electronically.
- It allows efficient storage, retrieval, and management of data.

Example

- | | |
|--------------|-----------------------------------|
| ▪ E-commerce | Products, Orders, Customers |
| ▪ Banking | Accounts, Transactions, Customers |
| ▪ Education | Students, Courses, Grades |

1.2. What is SQL?

- SQL - Structured Query Language
- SQL is a standard programming language used to create, read, update, and delete (CRUD) data in a relational database.
- SQL is used in almost every modern data system (PostgreSQL, MySQL etc.)

Key Capabilities

Operation	SQL Keyword	Description
Create	CREATE	Define tables, views, and databases
Read	SELECT	Retrieve data from tables
Update	UPDATE	Modify existing data
Delete	DELETE	Remove data

Example:

```
SELECT name, salary
FROM employees
WHERE department = 'Sales';
```



1.3. SQL vs NoSQL

Feature	SQL (Relational DB)	NoSQL (Non-Relational DB)
Data Model	Tables (Rows & Columns)	Documents, Key-Value, Graphs, Wide-Column
Schema	Fixed (Predefined)	Dynamic (Flexible)
Relationships	Strong (via Foreign Keys)	Weak or None
Query Language	SQL	No standard (MongoDB Query, etc.)
Scaling	Vertical (bigger machine)	Horizontal (more machines)
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, DynamoDB

- ✓ Use SQL when data is structured and relationships matter (banking, ERP).
- ✓ Use NoSQL when flexibility and speed matter (social media, IoT).

1.4. Tables, Rows, Columns

Sid	Sname	Email	Phone	City
101	Srinivas	srinivas@da.com	111	Hyderabad
102	sd	sd@da.com	222	Bangalore
103	ds	ds@da.com	333	Chennai
104	hello	hello@da.com	444	Pune
105	hai	hai@da.com	555	Mumbai



Table

- ✓ Table stores data in rows and columns.
- ✓ Each table represents an entity
- ✓ **Example:** students

Row

- ✓ Single record in the table.
- ✓ **Example:** One student details.

Column

- ✓ Field describing an attribute.
- ✓ **Example:** sid, name, city.

1.5. Types of Keys

- **4 Types of Keys**
 - a) Primary Key
 - b) Foreign Key
 - c) Candidate Key
 - d) Composite Key

a) Primary Key

- Uniquely identifies each record
- Ensure data integrity and prevent duplicates in relational databases.

Ex:

student_id in students table



b) Foreign Key

- Creates relationship between tables
- Maintains referential integrity

Ex:

student_id in students → courses

c) Candidate Key

- Any column that can uniquely identify a record — meaning it could serve as a primary key.
- Every table can have multiple candidate keys, but only one becomes the primary key.

Ex:

student_id, email, and phone in students table can all be unique
→ they are candidate keys.

d) Composite Key

- Combination of two or more columns used together to uniquely identify a record when a single column is not enough.

Ex:

(student_id, course_id) together form the Composite Key.

1.6. Basic Database Terminology

- a) Instance
- b) Schema
- c) Entity
- d) Record
- e) Attribute



a) Instance

- Instance is the actual data stored in the database at a particular moment.
- Think of it as: the current snapshot of data in all tables.

Ex:

Current data stored in the database right now

b) Schema

- Schema defines the structure of the database — how data is organized.
- Includes: table names, columns, data types, and relationships (keys).

Ex:

Database name → myda_db

Contains → tables (students, courses), columns, data types, and relationships.

c) Entity

- Entity is any real-world object about which data is stored in the database.
- Think of it as: a table in the database.

Ex:

students (table representing student entity)

courses (table representing course entity)

d) Record

- Record represents a single instance of an entity — one complete row in a table.
- Think of it as: one entry in the table.

Ex:

(101, 'Srinivas', 'sri@da.com', '12345', 'ML')

e) Attribute/Column

- Attribute is a property or characteristic of an entity.
- Think of it as: a column in a table.

Ex:

sid - 101

sname - Srinivas



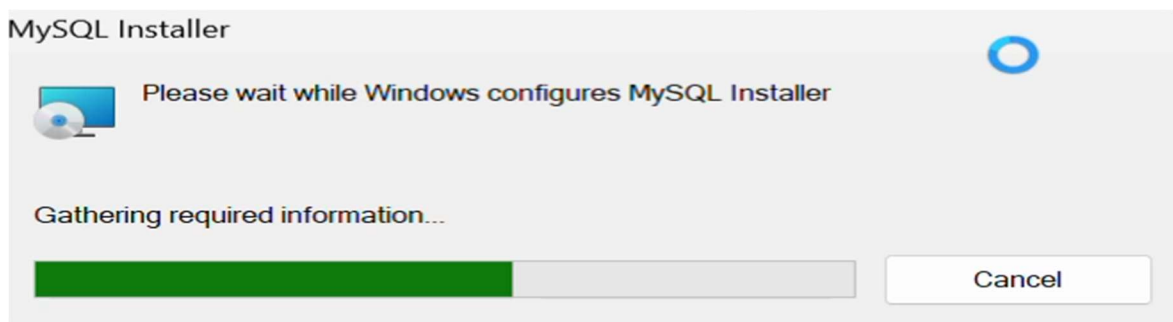
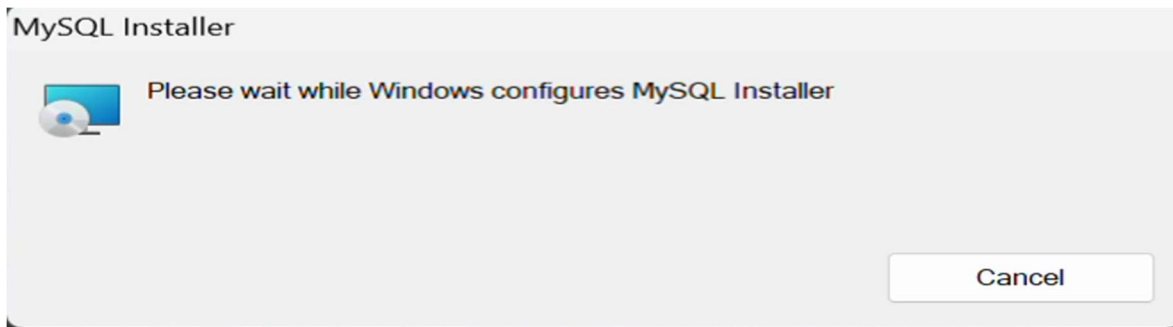
2. DB Setup and SQL Fundamentals

- 2.1. Installing MySQL Server
- 2.2. Installing MySQL Workbench
- 2.3. Work with MySQL Workbench
- 2.4. Creating a New Database (Schema)
- 2.5. Creating a Table
- 2.6. Inserting Data
- 2.7. Viewing Data
- 2.8. Updating Data
- 2.9. Deleting Data
- 2.10. ALTER Command
- 2.11. Dropping a Table
- 2.12. Truncating a Table
- 2.13. SQL Command Types

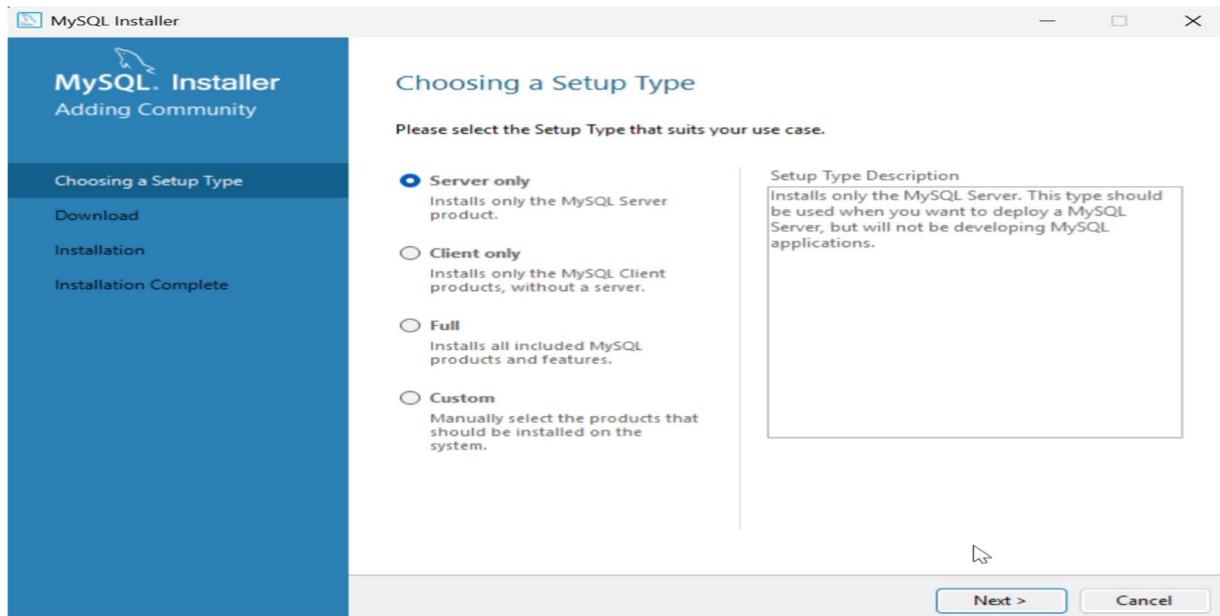


2.1. Installing MySQL Server

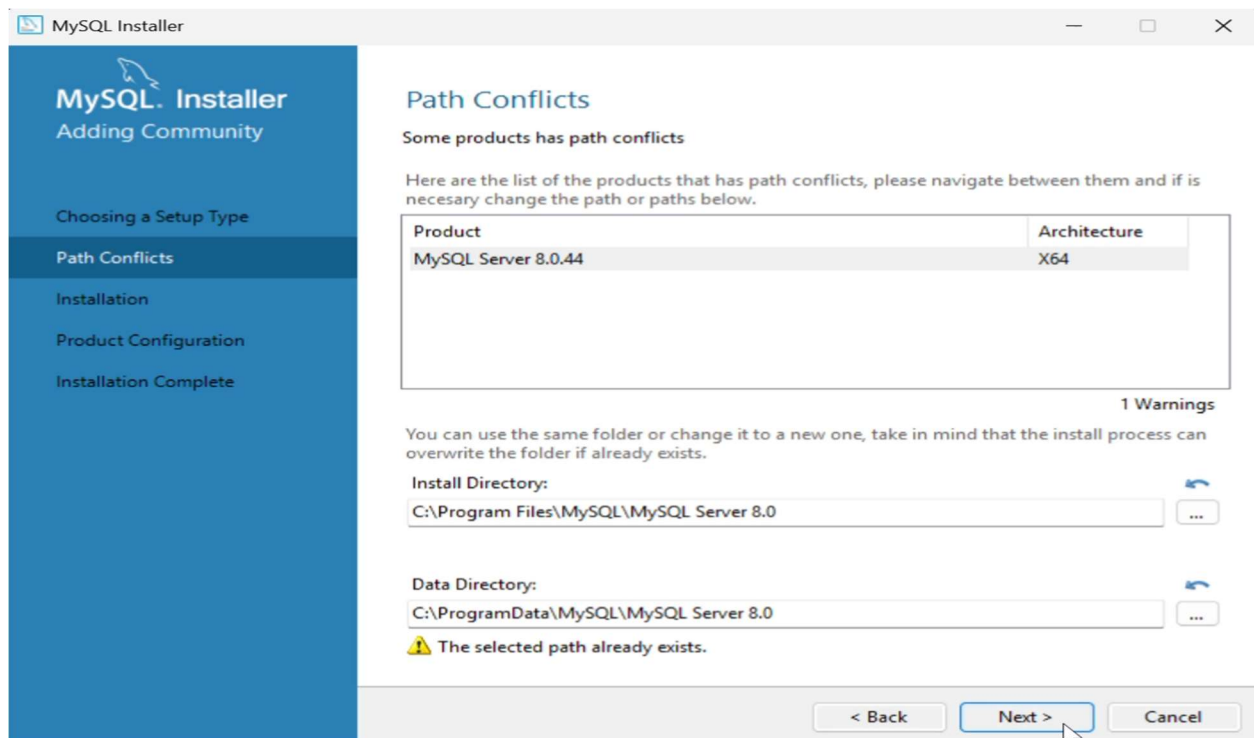
1) Double click on the “ MySQL-installer-community-8.0.44.0 ”.



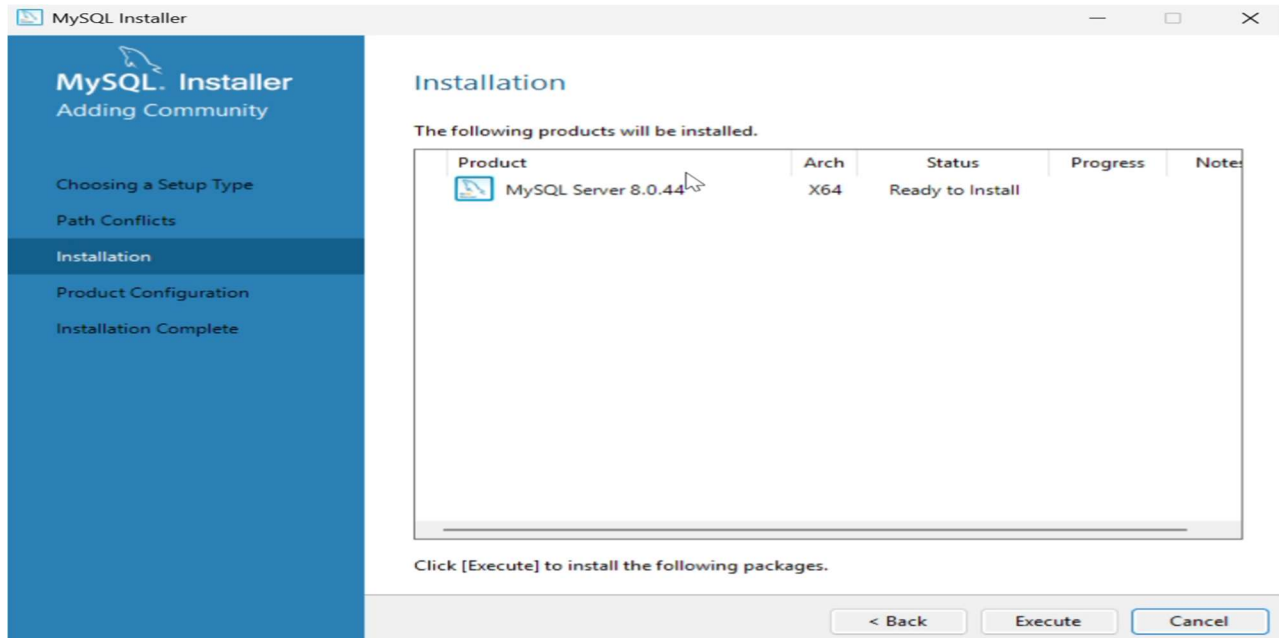
2) Select Server only and click on next button.



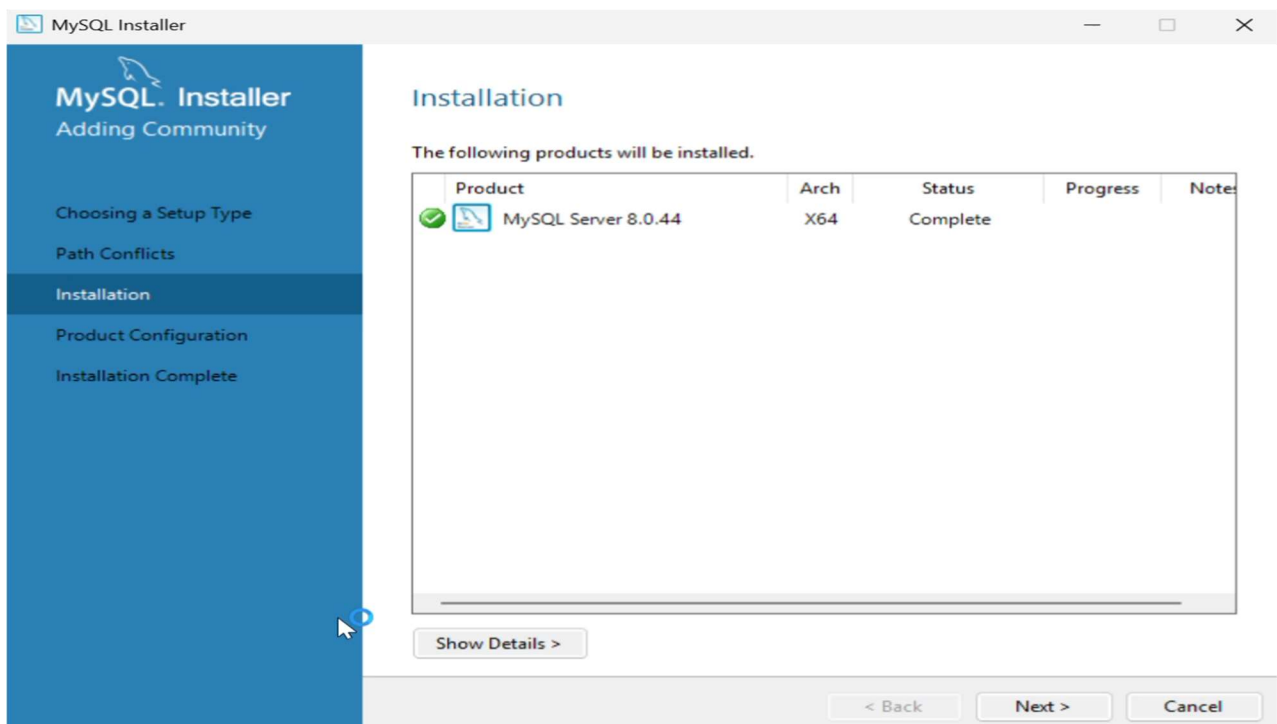
3) Set your path and click on next button.



4) Click on Execute button and wait...

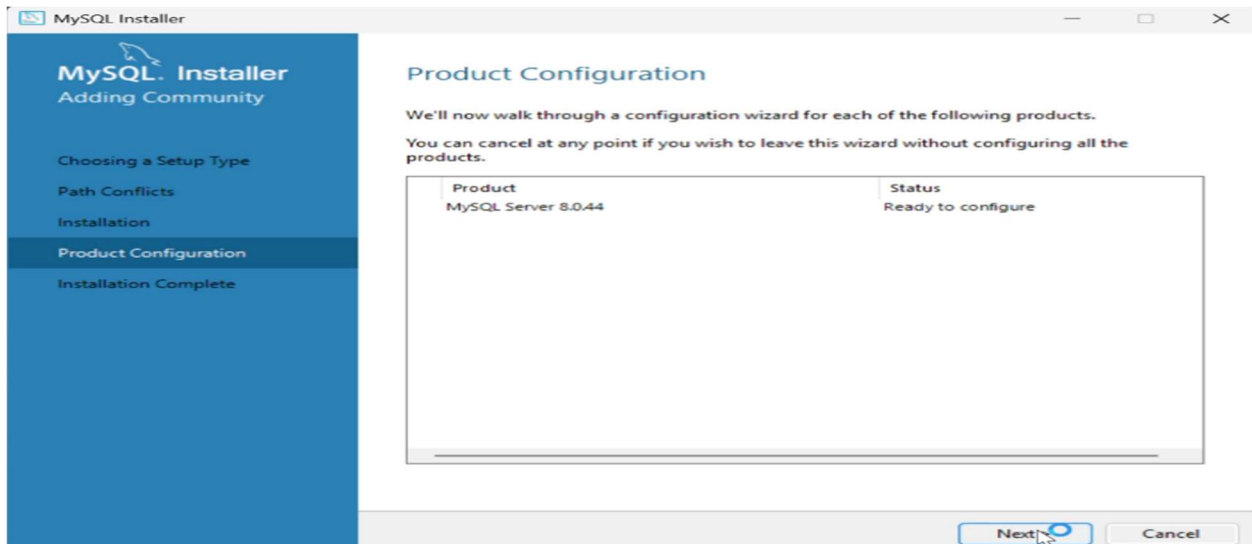


5) Click on next button.

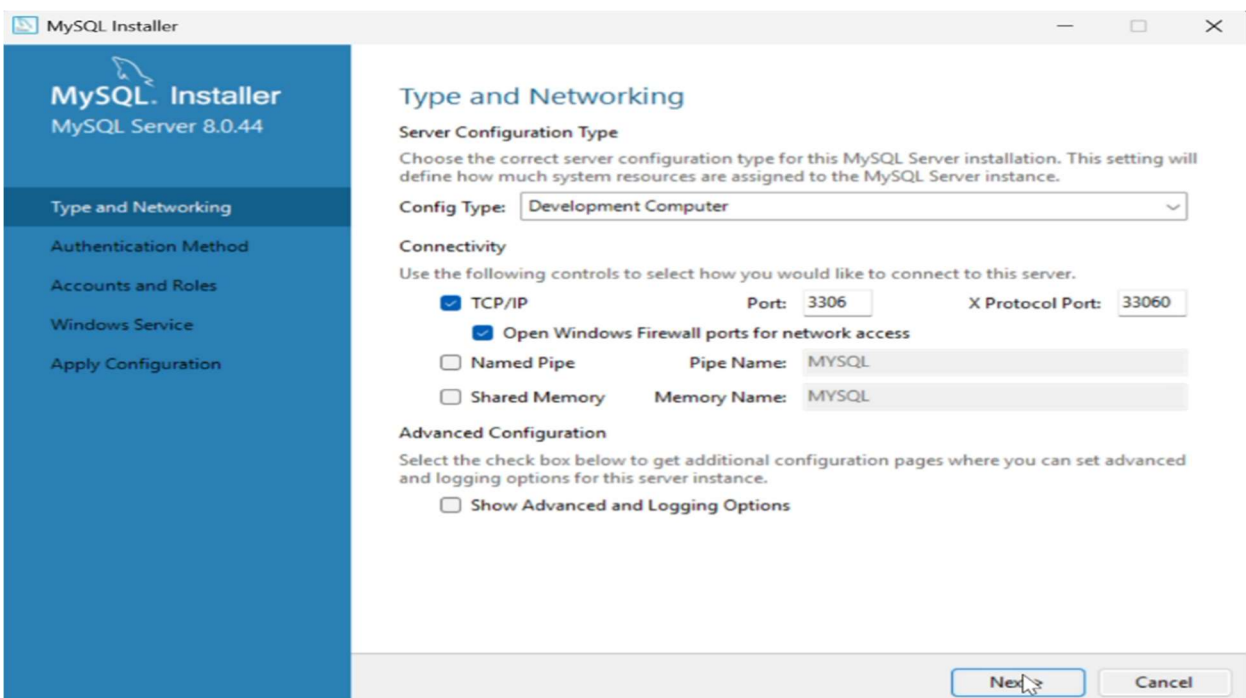




6) Click on next button.

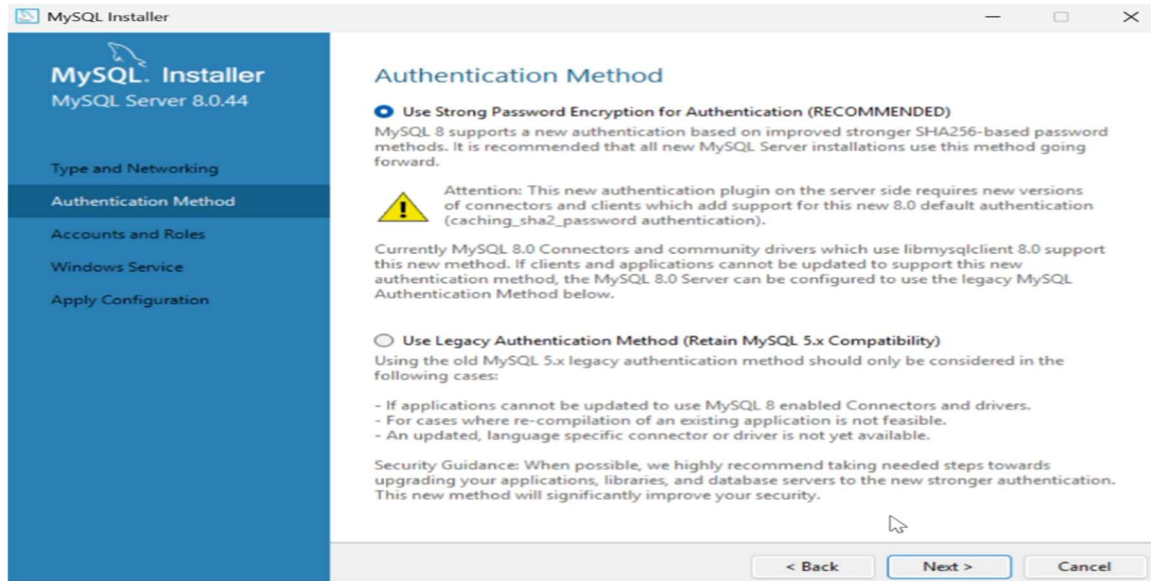


7) Click on next button.

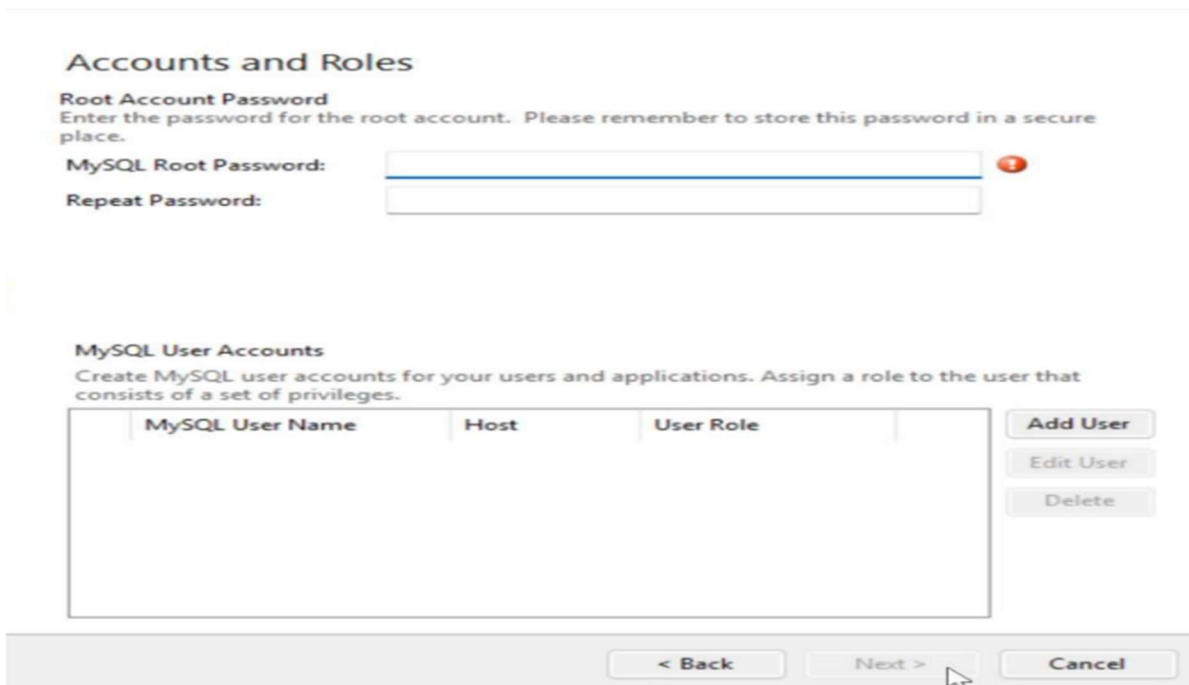




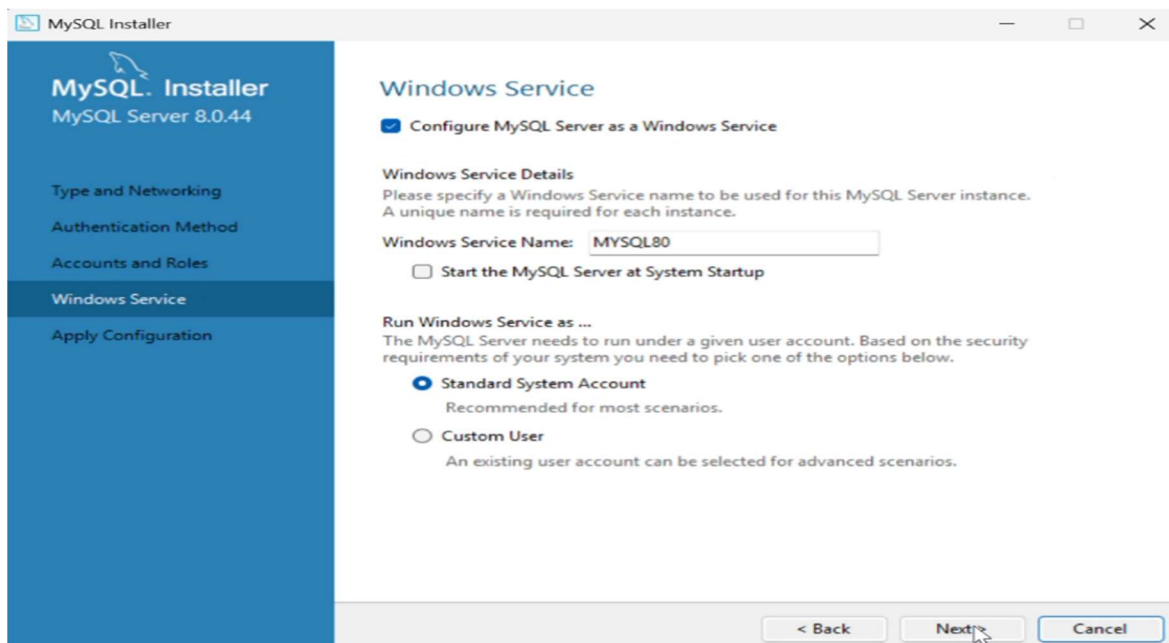
8) Click on next button.



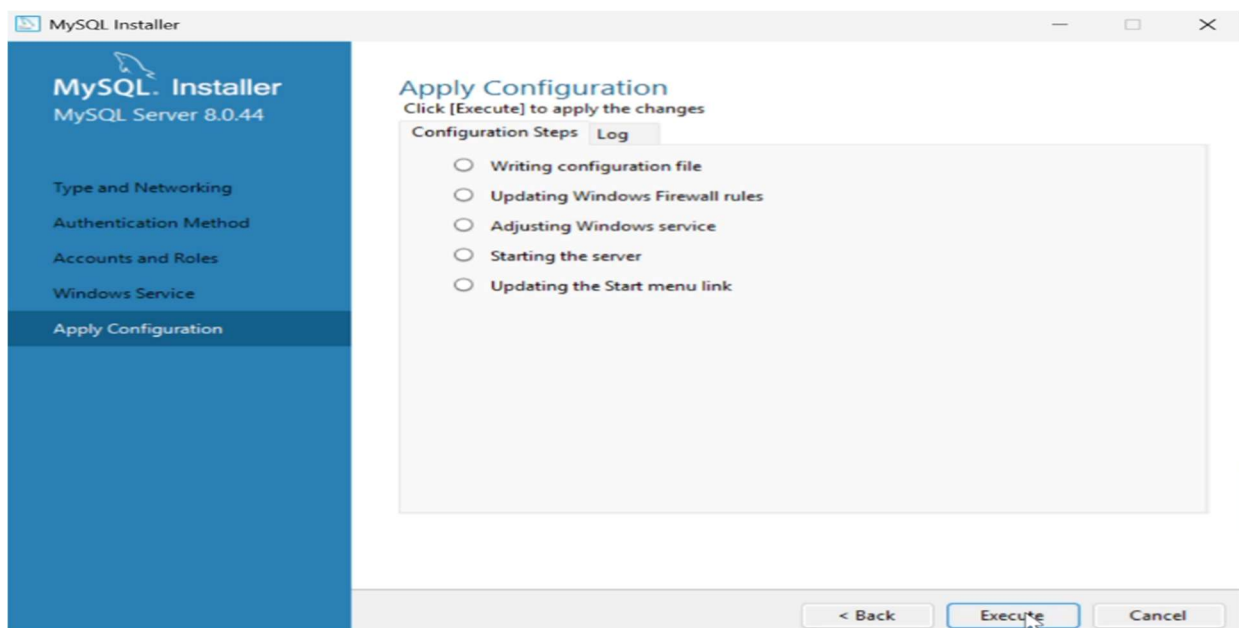
9) Enter Root Password and Click on next button.



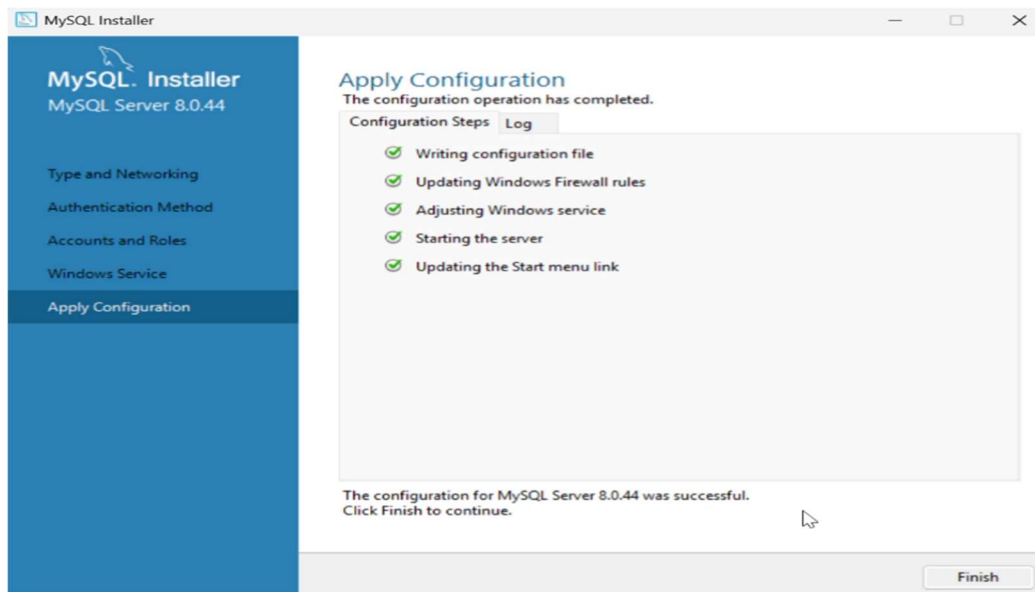
10) Click on next button.



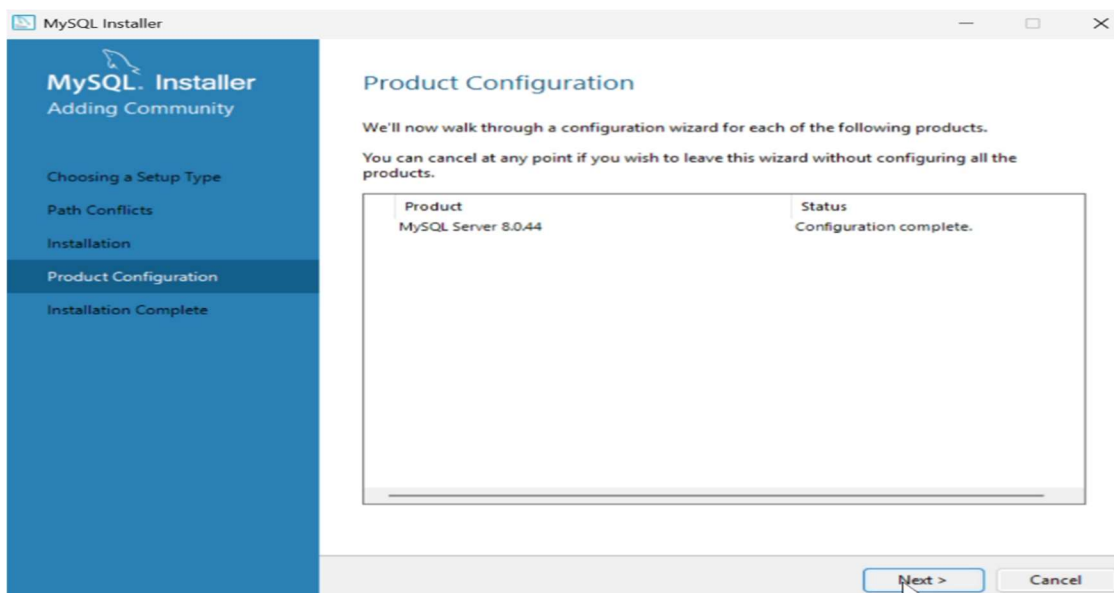
11) Click on Execute button.



12) Click on Finish button.

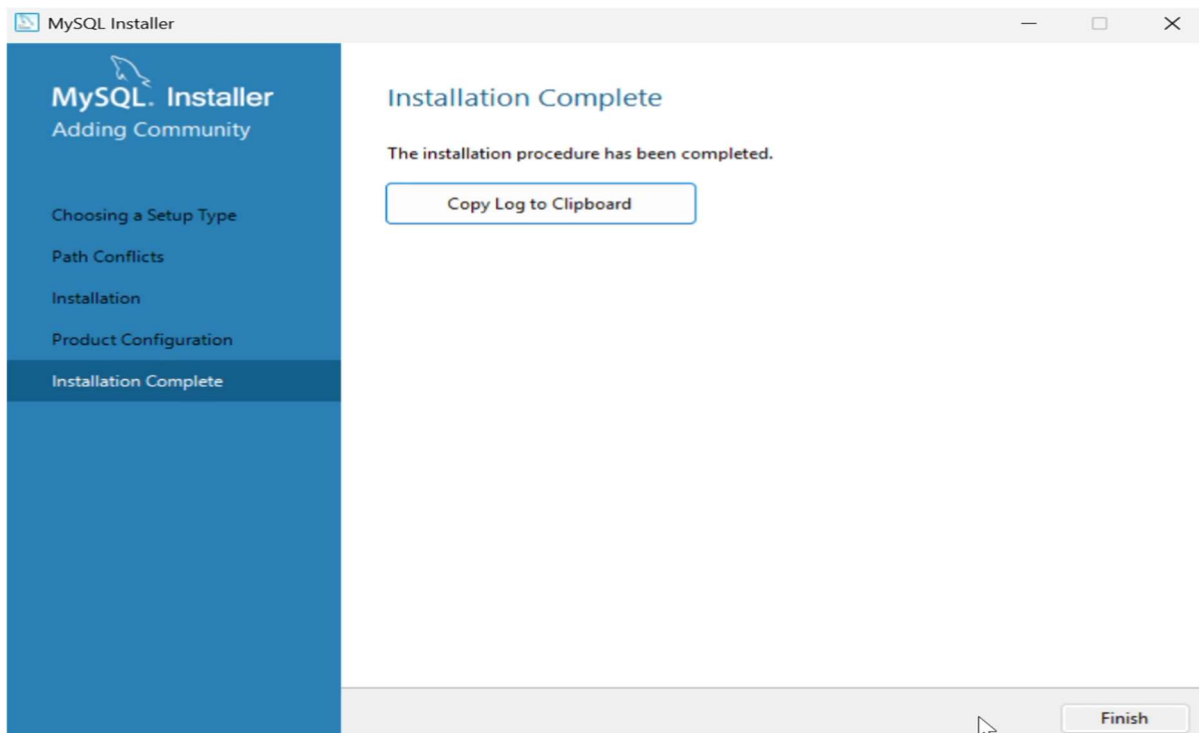


13) Click on next button.





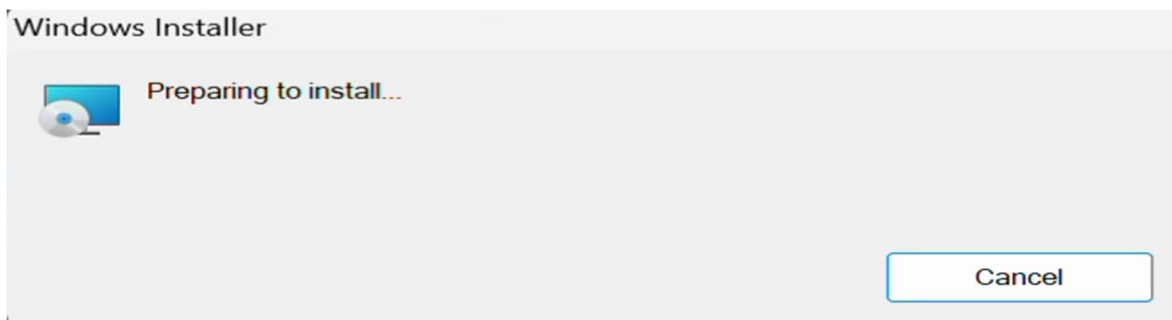
14) Click on Finish Button.



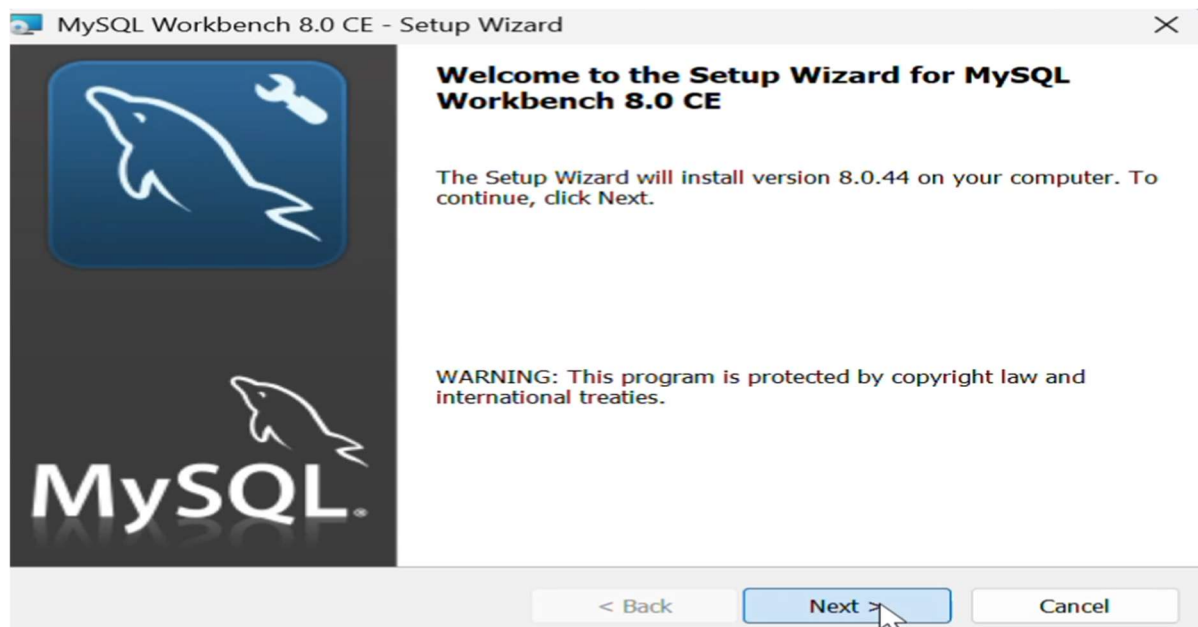


2.2. MySQL workbench 8.0 Installation

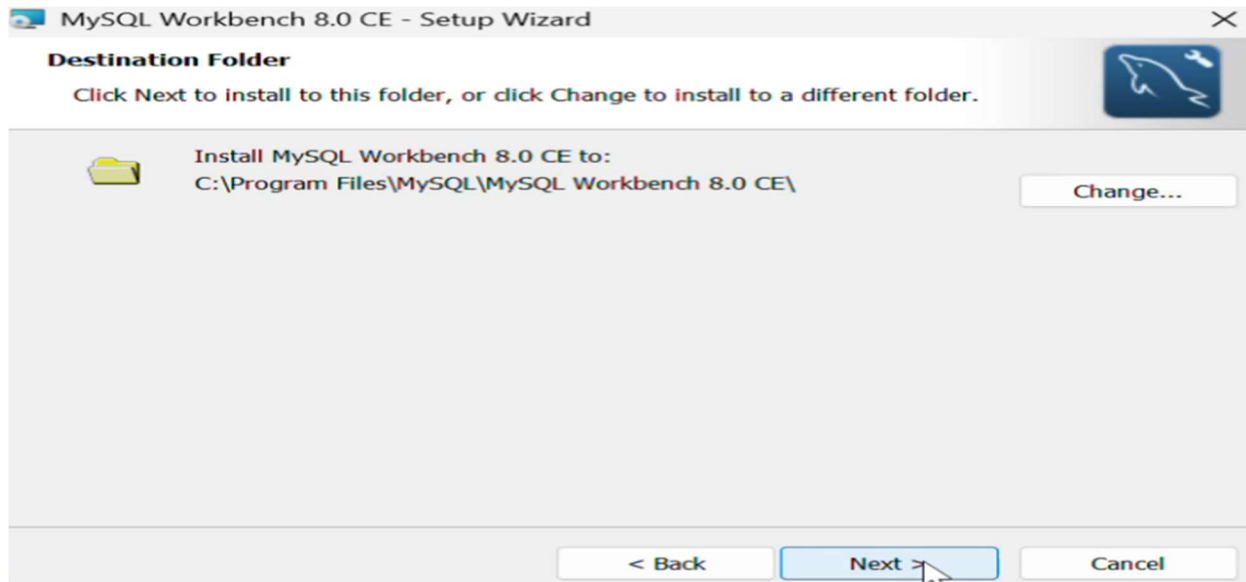
1) Click on “ MySQL-workbench-community-8.0.44-winx64 “ .



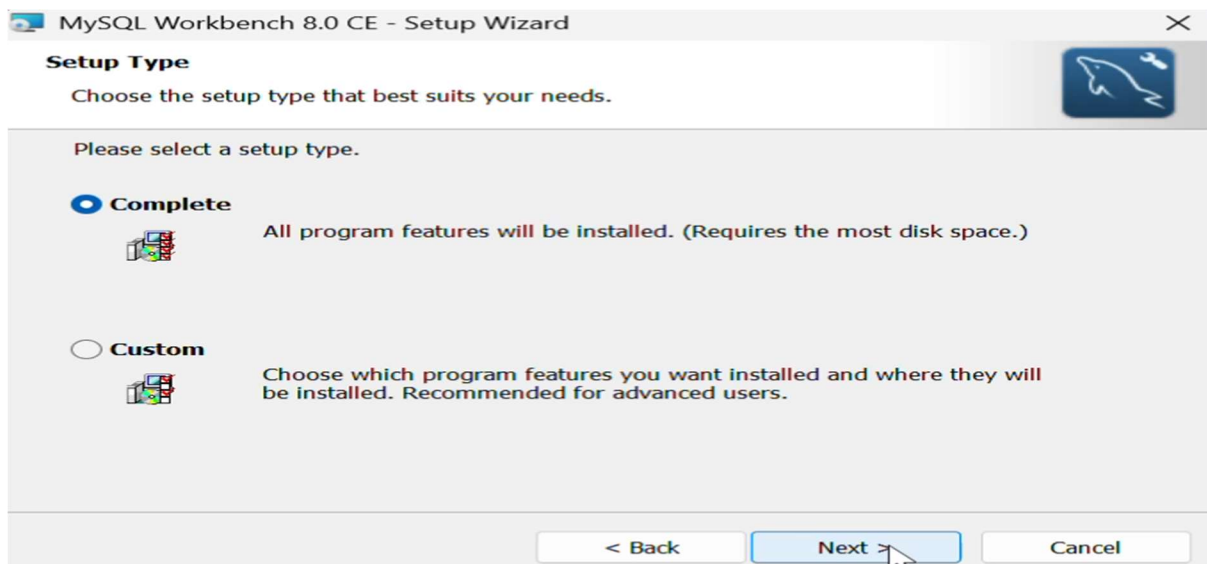
2) Click on Next Button.



3) Provide the Installation Location and Click on Next Button.

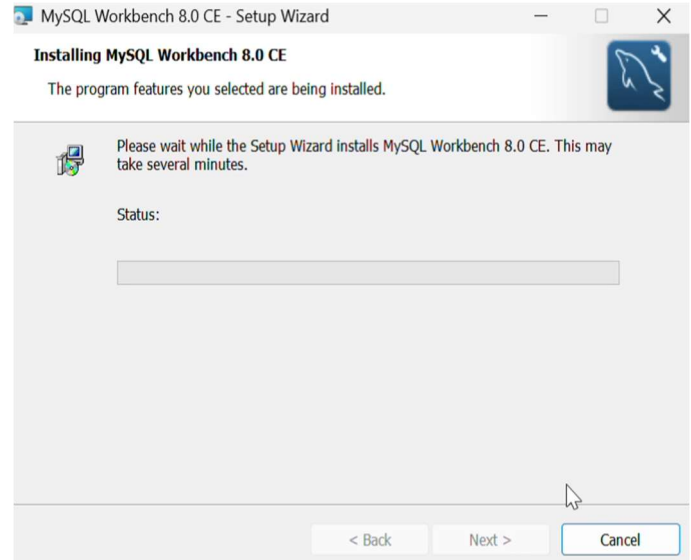
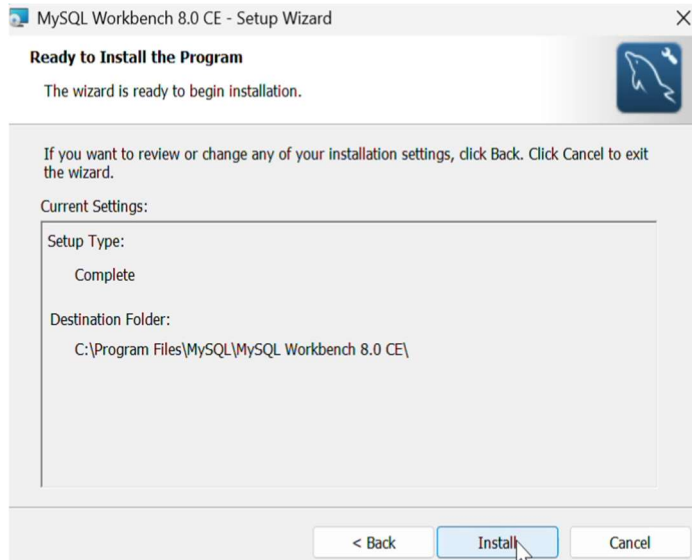


4) Choose Complete and Click on Next Button.

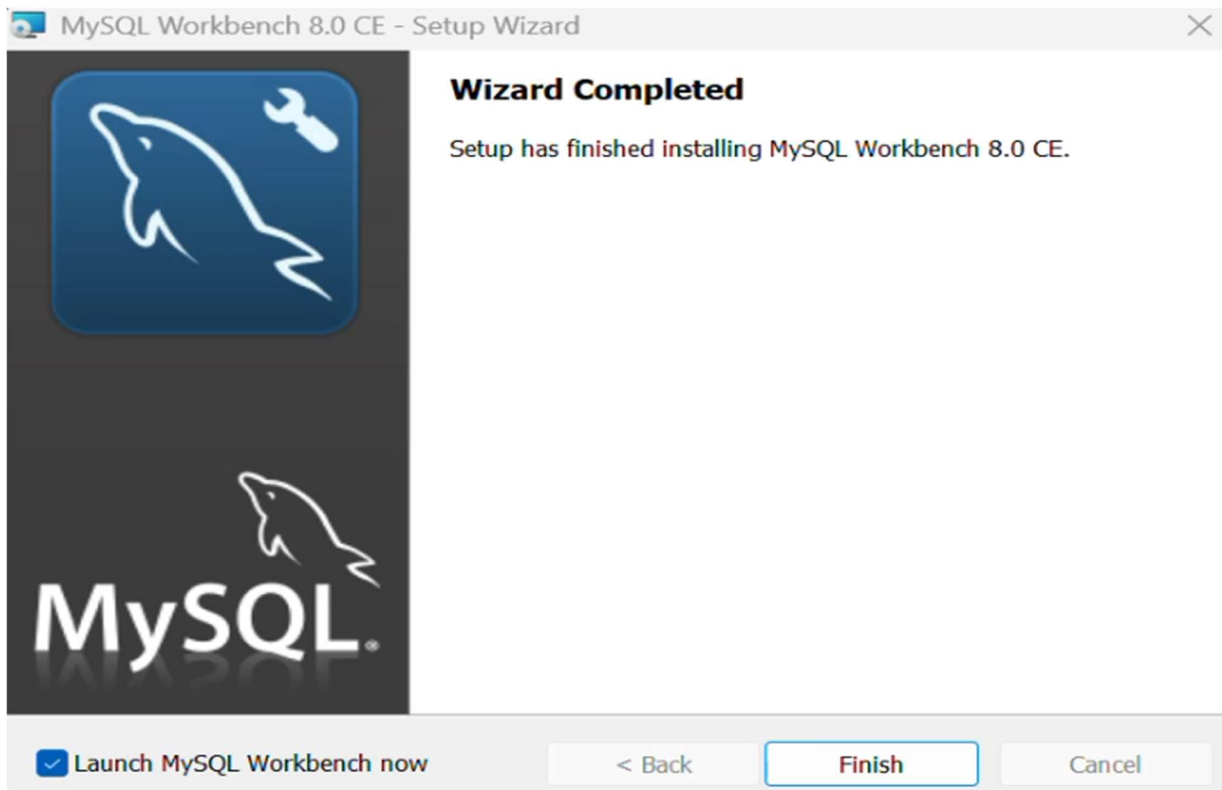




5) Click on Install Button and Wait it will Take 5 minutes Time.



6) Click on Finish.





2.3. Work with MySQL Workbench

- MySQL Workbench is a GUI tool used to connect to your MySQL Server and run SQL queries in the SQL Editor.

Steps:

- ✓ Launch MySQL Workbench.
- ✓ Click the connection (e.g., Local Instance MySQL80).
- ✓ Once connected, you can execute SQL commands inside the SQL Editor.

2.4. Creating a New Database (Schema)

- CREATE DATABASE is used to create a new database (also called a schema).
- USE selects which database you want to work with.

Ex:

```
CREATE DATABASE da_db;  
USE da_db;
```

2.5. Creating a Table

- CREATE TABLE is used to define the structure of a table
- — its columns, data types, and constraints.

Syntax:

```
CREATE TABLE <table_name> (  
    col_name DATATYPE(SIZE),  
    col_name DATATYPE(SIZE),  
    ...  
);
```



Example:

```
CREATE TABLE mystudents (  
    sid INT PRIMARY KEY,  
    sname CHAR(15),  
    email CHAR(15),  
    phone INT,  
    course CHAR(10),  
    fee DOUBLE  
);
```

2.6. Inserting Data

- INSERT INTO is used to add new records into a table.

Syntax:

```
INSERT INTO table_name (col1, col2, ...) VALUES (val1, val2, ...);
```

Examples:

```
INSERT INTO mystudents VALUES (101, 'sri', 'sri@myjlc', 12345, 'Java', 25000);  
INSERT INTO mystudents (sid, sname, email, phone) VALUES (102, 'vas', 'vas@myjlc', 54321);  
INSERT INTO mystudents (sid, sname, fee) VALUES (103, 'sd', 10000);  
INSERT INTO mystudents VALUES (104, 'ds', 'ds@myjlc', 98765, 'Python', 30000);  
INSERT INTO mystudents VALUES (105, 'hello', 'hello@myjlc', 67890, 'SQL', 20000);  
INSERT INTO mystudents VALUES (106, 'hai', 'hai@myjlc', 11223, 'ML', 40000);
```



2.7. Viewing Data

- SELECT is used to view or retrieve data from one or more tables.

Syntax:

```
SELECT * OR col1, col2
FROM table_name
WHERE condition
ORDER BY col_name
GROUP BY col_name
HAVING condition;
```

Examples:

SELECT * FROM mystudents;	-- Display all records
SELECT sname, course FROM mystudents;	-- View specific columns
SELECT * FROM mystudents WHERE course = 'Python';	-- Filter records

2.8. Updating Data

- UPDATE is used to modify existing records in a table.

Syntax:

```
UPDATE table_name
SET col1 = val1, col2 = val2
WHERE condition;
```

Examples:

```
UPDATE mystudents
SET email = 'sd@myjlc', phone = 9999, course = 'JavaFSD'
WHERE sid = 103;

UPDATE mystudents
SET course = 'DataScience'
WHERE sid = 105;
```



2.9. Deleting Data

- DELETE is used to remove one or more records from a table.
- Deletes the specified record(s) based on the given condition.

Syntax:

```
DELETE FROM table_name  
WHERE condition;
```

Example:

```
DELETE FROM mystudents WHERE sid = 103;  
DELETE FROM mystudents WHERE sname = 'hello';
```

2.10. ALTER Command

- ALTER is used to modify an existing table's structure
- — add, delete, or modify columns and constraints.

Syntax:

```
ALTER TABLE table_name  
ADD column_name DATATYPE(SIZE);
```

```
ALTER TABLE table_name  
MODIFY column_name DATATYPE(SIZE);
```

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

Example:

```
ALTER TABLE mystudents ADD city CHAR(15);           -- Adds a new column  
ALTER TABLE mystudents MODIFY sname VARCHAR(30);    -- Changes column size/type  
ALTER TABLE mystudents DROP COLUMN email;           -- Removes a column
```



2.11. Dropping a Table

- DROP TABLE permanently removes both data and structure.
- — cannot be rolled back.

Syntax:

```
DROP TABLE <table_name>;
```

Example:

```
DROP TABLE mystudents;
```

2.12. Truncating a Table

- TRUNCATE TABLE deletes all data inside the table but keeps the structure intact.
- — cannot be rolled back.

Syntax:

```
TRUNCATE TABLE <table_name>;
```

Example:

```
TRUNCATE TABLE mystudents;
```

Difference Between DROP, TRUNCATE, and DELETE

Command	Action	Keeps Table Structure	Rollback Possible
DROP	Deletes table + data	No	No
TRUNCATE	Deletes all data only	Yes	No
DELETE	Deletes rows	Yes	Yes



2.13. SQL Command Types

- a) DDL —Data Definition Language
- b) DML —Data Manipulation Language
- c) DQL —Data Query Language
- d) DCL —Data Control Language
- e) TCL — Transaction Control Language

a) DDL — Data Definition Language

- Used to define, modify, or remove the structure of database objects
- like tables, databases, views

Common Commands:

CREATE — Create new database objects

ALTER — Modify existing table structure

DROP — Delete database objects permanently

TRUNCATE — Remove all data but keep structure

b) DML — Data Manipulation Language

- Used to manipulate or modify data stored in tables.

Common Commands:

INSERT — Add new records into a table

UPDATE — Modify existing records

DELETE — Remove specific records

c) DQL — Data Query Language

- Used to retrieve or query data from one or more tables.

Common Commands:

SELECT — Fetch records from database tables



d) DCL — Data Control Language

- Used to control user access and permissions on database objects

Common Commands:

GRANT — Provide specific privileges to users

REVOKE — Remove previously granted privileges

e) TCL — Transaction Control Language

- Used to manage transactions and maintain data integrity
- Used during multiple DML operations.

Common Commands:

COMMIT — Save all changes permanently

ROLLBACK — Undo changes made in a transaction

SAVEPOINT — Create a temporary point to which you can roll back later



3. Data Types and Constraints

3.1. SQL Data Types

3.2. SQL Constraints

3.3. NOT NULL Constraint

3.4. UNIQUE Constraint

3.5. PRIMARY KEY Constraint

3.6. FOREIGN KEY Constraint

3.7. DEFAULT Constraint

3.8. CHECK Constraint

3.9. AUTO INCREMENT



3.1. SQL Data Types

- Data types define what kind of data can be stored in each column of a table.
- They help ensure data accuracy, consistency, and optimal memory usage.

Categories of SQL Data Types

- a) Numeric Data Types
- b) String (Character) Data Types
- c) Date & Time Data Types
- d) Boolean Data Type.

Usage	MySQL Data Types	Notes
Integer Values	INT, BIGINT, LONG	For whole numbers (e.g., ID, Age, Count)
Floating Values	FLOAT, DOUBLE	For decimal or real numbers (e.g., Price, Rating)
Alphanumeric / Text	CHAR(N), VARCHAR(N), TEXT	For strings, names, addresses, descriptions
Date & Time	DATE → 'YYYY-MM-DD'	For dates and timestamps
Large Documents	CLOB	For storing long text, logs
Binary Files (Images, Audio, Video)	BLOB, LONG BLOB	For binary content
Boolean Values	BOOLEAN / BOOL	For True / False flags (internally stored as 1 or 0)



3.2. SQL Constraints

- Constraints are rules applied on columns to maintain data accuracy, integrity, and consistency.
- They ensure that only valid data is inserted into a table.

Types of Constraints

- a) NOT NULL Constraint
- b) UNIQUE Constraint
- c) PRIMARY KEY Constraint
- d) FOREIGN KEY Constraint
- e) DEFAULT Constraint
- f) CHECK Constraint

3.3. NOT NULL Constraint

- Prevents storing NULL (missing) values in a column.
- Forces users to supply a valid value.
- Can be applied to multiple columns in the same table.

Example:

```
CREATE TABLE mycustomers1 (  
    cid INT NOT NULL,  
    cname CHAR(15) NOT NULL,  
    email CHAR(15) NOT NULL,  
    phone INT(10),  
    city CHAR(15)  
);
```

```
INSERT INTO mycustomers1(cid) VALUES(101);  
    -- # Error (cname & email cannot be NULL)
```

```
INSERT INTO mycustomers1(cid,cname,email) VALUES(101,'sri','sri@myjlc.com');  
    -- # Runs Successfully
```



3.4. UNIQUE Constraint

- Ensures that all values in a column are unique.
- Allows multiple NULLs unless combined with NOT NULL.
- Can be applied to multiple columns.

Example:

```
CREATE TABLE mycustomers2 (  
  cid INT NOT NULL UNIQUE,  
  cname CHAR(15) NOT NULL,  
  email CHAR(15) NOT NULL UNIQUE,  
  phone INT(10) UNIQUE,  
  city CHAR(15)  
);
```

```
INSERT INTO mycustomers2 VALUES(101,'sri','sri@da.com',12345,'Blore');  
-- # Runs Successfully
```

```
INSERT INTO mycustomers2 VALUES(102,'sd','sri@da.com',54321,'Blore');  
-- # Error (duplicate email)
```

```
INSERT INTO mycustomers2 VALUES(103,'ds','ds@da.com',NULL,'Blore');  
-- # Runs Successfully
```

3.5. PRIMARY KEY Constraint

- Combines NOT NULL + UNIQUE constraints.
- Ensures each record is uniquely identified.
- Only one Primary Key allowed per table.

Types

- a) Simple Primary Key → One column
- b) Composite Primary Key → Two or more columns



i. Simple Primary Key

Example:

```
CREATE TABLE mycustomers3 (  
    cid INT PRIMARY KEY,  
    cname CHAR(15) NOT NULL,  
    email CHAR(15) NOT NULL UNIQUE,  
    phone INT(10) NOT NULL UNIQUE,  
    city CHAR(15)  
);
```

```
INSERT INTO mycustomers3 VALUES(101,'sri','sri@myjlc.com',12345,'Blore');
```

```
SELECT * FROM mycustomers3 WHERE cid = 101;
```

ii) Composite Primary Key

```
CREATE TABLE students (  
    bid CHAR(3),  
    sid INT,  
    sname CHAR(15) NOT NULL,  
    email CHAR(15) NOT NULL UNIQUE,  
    phone INT(10) NOT NULL UNIQUE,  
    course CHAR(15),  
    PRIMARY KEY (bid, sid)  
);
```

```
INSERT INTO students VALUES('B1',101,'sri','sri@myjlc.com',111,'Java');
```

```
INSERT INTO students VALUES('B1',102,'sd','sd@myjlc.com',222,'Java');
```

```
SELECT * FROM students WHERE bid='B1' AND sid=101;
```



3.6. FOREIGN KEY Constraint

- Creates a relationship between two tables.
- Ensures referential integrity (child records must match parent records).
- Column in the child table refers to a Primary Key in the parent table.

Example:

```
CREATE TABLE mycustomers (  
    cid INT PRIMARY KEY,  
    cname CHAR(15) NOT NULL,  
    email CHAR(15) NOT NULL UNIQUE,  
    phone INT(10) UNIQUE,  
    city CHAR(15)  
);  
  
CREATE TABLE myaccounts (  
    mycid INT,  
    accno INT PRIMARY KEY,  
    atype CHAR(15) NOT NULL,  
    branch CHAR(15),  
    bal DOUBLE,  
    FOREIGN KEY (mycid) REFERENCES mycustomers(cid)  
);  
  
CREATE TABLE mytransactions (  
    myaccno INT,  
    txNumber INT PRIMARY KEY,  
    txDate DATE NOT NULL,  
    amount DOUBLE,  
    txType CHAR(2),  
    FOREIGN KEY (myaccno) REFERENCES myaccounts(accno)  
);  
  
INSERT INTO mycustomers VALUES(101,'sri','sri@jlc',12345,'Blore');  
  
INSERT INTO myaccounts VALUES(101,555,'SA','BTM',25000);  
  
INSERT INTO mytransactions VALUES(555,1,CURDATE(),2000,'Cr');
```



3.7. DEFAULT Constraint

- Assigns a default value if none is provided during insertion.
- Avoids NULL values when no explicit data is given.

Syntax:

column_name DATATYPE DEFAULT value;

Example:

```
CREATE TABLE students1 (  
    sid INT PRIMARY KEY,  
    sname CHAR(15) NOT NULL,  
    email CHAR(15) NOT NULL UNIQUE,  
    phone INT(10) NOT NULL UNIQUE,  
    course CHAR(15) DEFAULT 'Java',  
    city CHAR(15) DEFAULT 'Blore'  
);
```

```
INSERT INTO students1(sid,sname,email,phone) VALUES(101,'sri','sri@jlc',123);  
-- # course = 'Java' and city = 'Blore' are auto-filled
```

3.8. CHECK Constraint

- Defines a custom rule that values must satisfy before insertion.
- Enforces business rules at the database level.

Syntax:

column_name DATATYPE CHECK (condition);

Example:

```
CREATE TABLE students2 (  
    sid INT PRIMARY KEY,  
    sname CHAR(15) NOT NULL,  
    email CHAR(15) NOT NULL UNIQUE,  
    phone INT(10) UNIQUE,  
    course CHAR(15) DEFAULT 'Java',  
    totalfee DOUBLE CHECK (totalfee >= 25000)  
);
```

```
INSERT INTO students2 VALUES(101,'sri','sri@jlc',123,'Java',20000);  
-- # Error (fee < 25000)
```

```
INSERT INTO students2 VALUES(102,'sd','sd@jlc',456,'Java',30000);  
-- # Runs Successfully
```

3.9. AUTO INCREMENT

- AUTO INCREMENT feature is used to automatically generate unique numeric values for a column whenever a new record is inserted.
- It's most commonly applied to Primary Key columns like id, sid, or cid.

Key Points

- ✓ Automatically increases the value by 1 for each new row.
- ✓ No need to manually provide the value during insertion.
- ✓ Usually applied to INT columns with PRIMARY KEY.
- ✓ Each table can have only one AUTO_INCREMENT column.

Example:

```
CREATE TABLE mystudents_auto (  
    sid INT AUTO_INCREMENT PRIMARY KEY,  
    sname CHAR(15) NOT NULL,  
    email CHAR(30) NOT NULL UNIQUE,  
    phone INT(10) UNIQUE,  
    course CHAR(15) DEFAULT 'Java'  
);
```

```
INSERT INTO mystudents_auto (sname, email, phone, course)  
VALUES ('Srinivas', 'sri@da.com', 12345, 'ML');
```

```
INSERT INTO mystudents_auto (sname, email, phone, course)  
VALUES ('Hello', 'hello@da.com', 67890, 'SQL');
```



4. Filtering and Sorting

- Learn how to filter data using conditions and sort query results using SQL clauses and operators.

4.1. SELECT Statement

4.2. SQL Operators

4.3. Practice Lab — Filtering and Sorting

4.4 Common Query Examples

4.5. ORDER BY Clause



4.1. SELECT Statement

- Used to retrieve data from one or more tables.

Syntax:

```
SELECT * OR col1, col2, ...  
FROM table_name  
WHERE condition  
ORDER BY col_name [ASC|DESC]  
GROUP BY col_name  
HAVING condition;
```

4.2. SQL Operators

a) Arithmetic Operators

- Used for basic mathematical calculations.

+, -, *, /, MOD

b) Relational (Comparison) Operators

- Used to compare values in conditions.

=, != (or <>), >, <, >=, <=

c) Logical Operators

- Used to combine multiple conditions.

AND, OR, NOT

d) IN Operator

- Checks if a value matches any value in a list.

WHERE city IN ('Blore', 'Hyd', 'Pune');



e) BETWEEN Operator

- Checks if a value lies within a given range.

WHERE bal BETWEEN 8000 AND 20000;

f) LIKE Operator

- Used for pattern matching in text (with wildcards).
- % → matches any number of characters
- _ → matches a single character

WHERE sname LIKE 's%'; -- names starting with 's'

WHERE email LIKE '%@jlc'; -- emails ending with '@jlc'

g) IS NULL Operator

- Checks for NULL values.

WHERE phone IS NULL;

h) NOT Operator

- Negates a condition.

WHERE city NOT IN ('Blore', 'Hyd');



4.3. Practice Lab — Filtering and Sorting

Step 1: Create the Table called mystudents as follows

```
create table mystudents(  
  sid int(3) primary key,  
  sname char(15),  
  email char(15),  
  phone long,  
  city char(15),  
  course char(15),  
  bal double  
);
```

Step 2: Insert the Following Records

```
insert into mystudents values(101,'Sri','sri@jlc',123456,'Blore','Java',9000);  
insert into mystudents values(102,'Vas','Vas@jlc',654321,'Blore','Java',15000);  
insert into mystudents values(103,'ds','ds@gmail.com',1234,'Blore','DevOps',3000);  
insert into mystudents values(104,'sd','sd@gmail.com',4321,'Hyd','AWS',5000);  
insert into mystudents values(105,'hello','hello@jlc',5555,'Delhi','Java',8000);  
insert into mystudents values(106,'hai','hai@gmail.com',9999,'Blore','AWS',6000);  
insert into mystudents values(107,'aaa','aaa@jlc',1111,'Pune','DevOps',22000);  
insert into mystudents values(108,'bbb','bbb@jlc',2222,'Delhi','Java',7000);  
insert into mystudents values(109,'ccc','ccc@jlc',3333,'Blore','Java',20000);  
insert into mystudents values(110,'ddd','ddd@jlc',4444,'Hyd','Python',5000);  
insert into mystudents(sid,sname,email,course)  
  values(111,'eee','eee@gmail.com','Java');  
insert into mystudents(sid,sname,phone,course) values(112,'fff',5555,'Python');
```



Note : After inserting the records, table looks as follows

```
mysql> select * from mystudents;
```

sid	sname	email	phone	city	course	bal
101	sri	sri@jlc	123456	Blore	Java	9000
102	Vas	Vas@jlc	654321	Blore	Java	15000
103	ds	ds@gmail.com	1234	NULL	DevOps	3000
104	sd	sd@gmail.com	4321	Hyd	AWS	5000
105	hello	hello@jlc	5555	Delhi	Java	8000
106	hai	hai@gmail.com	9999	Blore	AWS	6000
107	aaa	aaa@jlc	1111	Pune	DevOps	22000
108	bbb	bbb@jlc	2222	Delhi	Java	7000
109	ccc	ccc@jlc	3333	Blore	Java	20000
110	ddd	ddd@jlc	4444	Hyd	Python	5000
111	eee	eee@gmail.com	NULL	NULL	Java	NULL
112	fff	NULL	5555	NULL	Python	NULL

12 rows in set (0.16 sec)

4.4 Common Query Examples

Q1) Display full info of all students

```
SELECT * FROM mystudents;
```

Q2) Display name, phone, and balance of all students

```
SELECT sname, phone, bal FROM mystudents;
```

Q3) Display name and course of all students

```
SELECT sname, course FROM mystudents;
```



Q4) Display student whose sid = 105

```
SELECT * FROM mystudents  
WHERE sid = 105;
```

Q5) Display students from Bangalore

```
SELECT * FROM mystudents  
WHERE city = 'Blore';
```

Q6) Students staying in Hyd or Blore

```
SELECT * FROM mystudents  
WHERE city = 'Blore' OR city = 'Hyd';
```

```
-- or  
SELECT * FROM mystudents  
WHERE city IN ('Blore','Hyd');
```

Q7) Students staying in Hyd, Delhi, or Pune

```
SELECT * FROM mystudents  
WHERE city IN ('Hyd','Delhi','Pune');
```

Q8) Students NOT staying in Blore or Hyd

```
SELECT * FROM mystudents  
WHERE city NOT IN ('Blore','Hyd');
```

Q9) Students whose balance > 5000

```
SELECT * FROM mystudents  
WHERE bal > 5000;
```



Q10) Students whose balance ≤ 5000

```
SELECT * FROM mystudents  
WHERE bal <= 5000;
```

Q11) Students whose balance is between 8000 and 20000

```
SELECT * FROM mystudents  
WHERE bal BETWEEN 8000 AND 20000;
```

Q12) Students whose balance > 8000 and < 20000

```
SELECT * FROM mystudents  
WHERE bal > 8000 AND bal < 20000;
```

Q13) Students whose balance ≤ 8000 or ≥ 20000

```
SELECT * FROM mystudents  
WHERE bal <= 8000 OR bal >= 20000;
```

Q14) Students whose balance is NOT between 8000 and 20000

```
SELECT * FROM mystudents  
WHERE bal NOT BETWEEN 8000 AND 20000;
```

Q15) Students whose balance ≥ 15000 and city = 'Blore'

```
SELECT * FROM mystudents  
WHERE bal >= 15000 AND city = 'Blore';
```

Q16) Students whose balance ≥ 5000 , city = 'Blore', and course = 'AWS'

```
SELECT * FROM mystudents  
WHERE bal >= 5000 AND city = 'Blore' AND course = 'AWS';
```



Q17) Students who joined Java, AWS, or Python

```
SELECT * FROM mystudents  
WHERE course IN ('Java','AWS','Python');
```

Q18) Bangalore students who joined Java, AWS, or Python

```
SELECT * FROM mystudents  
WHERE city = 'Blore' AND course IN ('Java','AWS','Python');
```

Q19) Students from Blore or Delhi, joined Java/AWS, and balance between 8000–20000

```
SELECT * FROM mystudents  
WHERE course IN ('Java','AWS')  
AND city IN ('Blore','Delhi')  
AND bal BETWEEN 8000 AND 20000;
```

4.5. ORDER BY Clause

- Used to sort query results in ascending (ASC) or descending (DESC) order.

Syntax:

```
SELECT *  
FROM table_name  
ORDER BY column_name [ASC|DESC];
```

Q20) Display all students sorted by name (ascending)

```
SELECT *  
FROM mystudents  
ORDER BY sname;
```

Default order is ascending (A → Z).

Q21) Display all students sorted by balance (descending)

```
SELECT *  
FROM mystudents  
ORDER BY bal DESC;
```

Highest balance first.

Q22) Display all students sorted by city (ascending) and balance (descending)

```
SELECT *  
FROM mystudents  
ORDER BY city ASC, bal DESC;
```

Within each city, students appear in descending order of balance.

Q23) Display all students sorted by course (ascending) and name (ascending)

```
SELECT *  
FROM mystudents  
ORDER BY course, sname;
```

Groups students by course and sorts alphabetically by name.



5. Aggregation and Grouping

- Learn how to summarize and analyze data in SQL using aggregate functions, GROUP BY, and HAVING clauses.

5.1. What are Aggregate Functions?

5.2. Practice Lab — Aggregation and Grouping

5.3. GROUP BY Clause

5.4. HAVING Clause

5.5. Handling NULLs in Aggregates



5.1. What are Aggregate Functions?

- Aggregate functions perform calculations on a set of values and return one result per group or entire table.

Function	Description	Example
COUNT()	Counts total number of rows	COUNT(*)
SUM()	Adds up numeric values	SUM(bal)
AVG()	Finds average of numeric values	AVG(bal)
MIN()	Finds smallest value	MIN(bal)
MAX()	Finds largest value	MAX(bal)

5.2. Practice Lab — Aggregate Functions

Step 1: Create the Table called jlcstudents as follows

```
create table jlcstudents(  
  sid int(3) primary key,  
  sname char(15),  
  city char(15),  
  course char(15),  
  feepaid double,  
  feebal double,  
  status char(10)  
);
```



Step 2: Insert the Following Records

```
insert into jlcstudents values(101,'sri','Blore','Java',10000,10000,'Active');
insert into jlcstudents values(102,'vas','Hyd','AWS',5000,15000,'Active');
insert into jlcstudents values(103,'sd','Delhi','Java',12000,8000,'Active');
insert into jlcstudents values(104,'ds','Blore','Python',8000,12000,'Active');
insert into jlcstudents values(105,'aa','Pune','Java',15000,5000,'InActive');
insert into jlcstudents values(106,'bb','Blore','Java',10000,10000,'InActive');
insert into jlcstudents values(107,'cc','Delhi','Python',12000,8000,'InActive');
insert into jlcstudents values(108,'dd','Blore','Java',8000,12000,'Active');
insert into jlcstudents values(109,'hello','Pune','Python',5000,15000,'Active');
insert into jlcstudents values(110,'hai','Delhi','Java',15000,15000,'Active');

insert into jlcstudents(sid,sname,course,feepaid,feebal)
values(111,'abc','Python',5000,15000);

insert into jlcstudents(sid,sname,course,feepaid,feebal)
values(112,'xyz','Java',5000,15000);
```

Note : After inserting the records, table looks as follows

```
mysql> select * from jlcstudents;
+----+-----+-----+-----+-----+-----+-----+
| sid | sname | city  | course | feepaid | feebal | status |
+----+-----+-----+-----+-----+-----+-----+
| 101 | sri   | Blore | Java   | 10000   | 10000   | Active |
| 102 | vas   | Hyd   | AWS    | 5000    | 15000   | Active |
| 103 | sd    | Delhi | Java   | 12000   | 8000    | Active |
| 104 | ds    | Blore | Python | 8000    | 12000   | Active |
| 105 | aa    | Pune  | Java   | 15000   | 5000    | InActive |
| 106 | bb    | Blore | Java   | 10000   | 10000   | InActive |
| 107 | cc    | Delhi | Python | 12000   | 8000    | InActive |
| 108 | dd    | Blore | Java   | 8000    | 12000   | Active |
| 109 | hello | Pune  | Python | 5000    | 15000   | Active |
| 110 | hai   | Delhi | Java   | 15000   | 15000   | Active |
| 111 | abc   | NULL  | Python | 5000    | 15000   | Active |
| 112 | xyz   | NULL  | Java   | 5000    | 15000   | Active |
+----+-----+-----+-----+-----+-----+-----+
12 rows in set (0.00 sec)
```



Step 3: Practice the Following Queries

Q1) COUNT() — Count total number of rows

```
SELECT COUNT(*) AS total_students FROM mystudents;  
# Counts all rows in the table.
```

```
SELECT COUNT(city) AS total_cities FROM mystudents;  
# Counts non-null city entries.
```

Q2) SUM() — Add all balances

```
SELECT SUM(bal) AS total_balance FROM mystudents;  
# Returns total balance across all students.
```

Q3) AVG() — Find average balance

```
SELECT AVG(bal) AS avg_balance FROM mystudents;  
# Returns average balance of all students.
```

Q4) MIN() — Find minimum balance

```
SELECT MIN(bal) AS lowest_balance FROM mystudents;
```

Q5) MAX() — Find maximum balance

```
SELECT MAX(bal) AS highest_balance FROM mystudents;
```



5.3. GROUP BY Clause

- GROUP BY groups rows having same values in one or more columns and allows aggregate functions to be applied to each group.

Syntax

```
SELECT column_name, AGG_FUNC(column_name)
FROM table_name
GROUP BY column_name;
```

Q6) Total balance per course

```
SELECT course, SUM(bal) AS total_balance
FROM mystudents
GROUP BY course;
```

Q7) Average balance per city

```
SELECT city, AVG(bal) AS avg_balance
FROM mystudents
GROUP BY city;
```

Q8) Count students per course

```
SELECT course, COUNT(*) AS total_students
FROM mystudents
GROUP BY course;
```



5.4. HAVING Clause

- HAVING filters the results after grouping — it works like WHERE, but can be used only with aggregate functions.

Syntax

```
SELECT column_name, AGG_FUNC(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

Q9) Show courses with total balance > 10000

```
SELECT course, SUM(bal) AS total_balance
FROM mystudents
GROUP BY course
HAVING SUM(bal) > 10000;
```

Q10) Cities with average balance $\geq$ 8000

```
SELECT city, AVG(bal) AS avg_balance
FROM mystudents
GROUP BY city
HAVING AVG(bal) >= 8000;
```

Q11) Multiple conditions

```
SELECT course, COUNT(*) AS total_students, SUM(bal) AS total_balance
FROM mystudents
GROUP BY course
HAVING COUNT(*) > 2 AND SUM(bal) > 10000;
```



5.5. Handling NULLs in Aggregates

- Aggregate functions like SUM(), AVG(), MIN(), MAX(), and COUNT(column)
- ignore NULL values — except COUNT(*), which counts all rows, including NULLs.

Q12) Compare COUNT(column) vs COUNT(*)

```
SELECT
    COUNT(phone) AS valid_phones,
    COUNT(*) AS total_records
FROM mystudents;
```

COUNT(phone) → counts only non-NULL phone numbers.

COUNT(*) → counts all rows in the table.

Q13) Using COALESCE() to handle NULL values in aggregation

```
SELECT
    AVG(COALESCE(bal, 0)) AS avg_balance
FROM mystudents;
```

COALESCE(bal, 0) replaces any NULL balance with 0 before averaging.

Useful when you want to treat missing numeric values as zero.

Q14) Find total and average balance per course, replacing NULL balances with 0

```
SELECT
    course,
    SUM(COALESCE(bal, 0)) AS total_balance,
    AVG(COALESCE(bal, 0)) AS avg_balance
FROM mystudents
GROUP BY course;
```



Ensures courses with NULL balances are still included.

COALESCE(bal, 0) treats missing balances as zero before aggregation.

Q15) Count students per city, replacing missing city names with “Unknown”

```
SELECT
    COALESCE(city, 'Unknown') AS city_name,
    COUNT(*) AS student_count
FROM mystudents
GROUP BY COALESCE(city, 'Unknown');
```

Groups NULL city values under “Unknown”.

Ensures no records are skipped due to missing city information.



6. Joins and Relationships

- Understand how to combine data from multiple tables using relationships and different types of SQL joins.

6.1. Table Relationships

6.2. What is a JOIN?

6.3. Types of Joins

6.4. Practice Lab – Joins

6.5. Inner Join (Equi Join)

6.6. Left Outer Join

6.7. Right Outer Join

6.8. Full Outer Join

6.9. JOINS with Extra Conditions

6.10. Self Join

6.11. Cross Join



6.1. Table Relationships

- Before performing joins, tables must have relationships through a common column (key).
- There are three main relationship types in relational databases
 - a) One-to-One Relationship
 - b) One-to-Many Relationship
 - b) Many-to-Many Relationship

a) One-to-One:

- Each record in Table A matches exactly one record in Table B.

Examples:

- 1) Employee ↔ Employee_Address
 - ✓ Each employee has exactly one address record.
- 2) User ↔ User_Profile
 - ✓ Each user in the system has one unique profile record
- 3) Citizen ↔ Passport
 - ✓ Each citizen has only one passport, and each passport belongs to one citizen.

b) One-to-Many

- One record in Table A can relate to multiple records in Table B.

Examples:

- 1) Customer → Accounts
 - ✓ One customer can have multiple bank accounts.
- 2) Teacher → Students
 - ✓ One teacher teaches multiple students,
 - ✓ but each student belongs to one teacher.
- 3) Department → Employees
 - ✓ One department has many employees,
 - ✓ but each employee belongs to one department.



c) Many-to-Many

- Multiple records in Table A can relate to multiple records in Table B.
- This relationship requires a third table to link them.

Examples:

1) Students ↔ Courses via Enrollments

- ✓ A student can take many courses; a course can have many students.

2) Products ↔ Orders via OrderDetails

- ✓ An order can contain many products, and a product can appear in many orders.

3) Doctors ↔ Patients via Appointments

- ✓ A doctor can treat many patients; a patient can visit many doctors.

6.2. What is a JOIN?

- Joins are used to fetch data from two or more tables based on a related column between them.
- The relationship is usually defined using Primary Key–Foreign Key.
- Joins help in viewing combined data in a single query.

6.3. Types of Joins

- Here are the types of joins in SQL:
 - a) INNER JOIN (Equi Join)
 - b) LEFT OUTER JOIN
 - c) RIGHT OUTER JOIN
 - d) FULL OUTER JOIN
 - e) SELF JOIN
 - f) CROSS JOIN



6.4. Practice Lab – Joins

Step 1: Create Tables

```
CREATE TABLE customers (  
    cid INT(3) PRIMARY KEY,  
    cname CHAR(15) NOT NULL,  
    email CHAR(15) NOT NULL UNIQUE,  
    phone INT(10) NOT NULL UNIQUE  
);
```

```
CREATE TABLE accounts (  
    mycid INT(3),  
    accno INT(5) PRIMARY KEY,  
    atype CHAR(2) NOT NULL,  
    bal DOUBLE NOT NULL  
);
```

```
CREATE TABLE address (  
    mycid INT(3),  
    addid INT(3) PRIMARY KEY,  
    street CHAR(15) NOT NULL,  
    city CHAR(15) NOT NULL,  
    state CHAR(15) NOT NULL  
);
```

Step 2: Insert the Following Records

```
insert into customers values(101,'sri','sri@jlc',111);  
insert into customers values(102,'vas','vas@jlc',222);  
insert into customers values(103,'sd','sd@jlc',333);  
insert into customers values(104,'ds','ds@jlc',444);  
insert into customers values(105,'hello','hello@jlc',555);  
insert into customers values(106,'hai','hai@jlc',666);  
insert into customers values(107,'aaa','aaa@jlc',777);  
insert into customers values(108,'bbb','bbb@jlc',888);  
insert into customers values(109,'ccc','ccc@jlc',999);
```



```
insert into accounts values(101,12345,'SA',5000);  
insert into accounts values(102,12346,'SA',15000);  
insert into accounts values(103,12347,'SA',25000);  
insert into accounts values(107,12348,'SA',3000);  
insert into accounts values(108,12349,'SA',13000);  
insert into accounts values(109,12350,'SA',18000);
```

```
insert into address values(101,1,'BTM','Blore','KA');  
insert into address values(102,2,'MHA','Blore','KA');  
insert into address values(103,3,'P1','Pune','MH');  
insert into address values(104,4,'D1','Delhi','Delhi');  
insert into address values(109,5,'D2','Delhi','Delhi');  
insert into address values(110,6,'P2','Pune','MH');  
insert into address values(111,7,'H1','Hyd','TG');  
insert into address values(112,8,'pp','Patna','BR');
```



6.5. Inner Join (Equi Join)

- Returns only matching records from both tables.

Syntax:

```
SELECT columns  
FROM table1  
INNER JOIN table2  
ON table1.common_col = table2.common_col;
```

Examples

Q1) Customers with matching accounts

```
SELECT cid, cname, phone, accno, bal  
FROM customers c  
INNER JOIN accounts a  
ON c.cid = a.mycid;
```

Q2) Customers with matching address

```
SELECT cid, cname, phone, city, state  
FROM customers c  
INNER JOIN address ad  
ON c.cid = ad.mycid;
```

Q3) Combine customers, accounts, and address

```
SELECT cid, cname, phone, accno, bal, city, state  
FROM customers c  
INNER JOIN accounts a ON c.cid = a.mycid  
INNER JOIN address ad ON c.cid = ad.mycid;
```



6.6. Left Outer Join

- Returns all records from the left table and matching records from the right table.
- Non-matching rows show NULL for right-side columns.

Syntax

```
SELECT columns  
FROM table1  
LEFT JOIN table2  
ON table1.common_col = table2.common_col;
```

Examples

Q1) Customers and their accounts (include customers without accounts)

```
SELECT c.cid, c.cname, a.accno, a.bal  
FROM customers c  
LEFT JOIN accounts a ON c.cid = a.mycid;
```

Q2) Customers with their addresses (include customers without address)

```
SELECT c.cid, c.cname, ad.city, ad.state  
FROM customers c  
LEFT JOIN address ad ON c.cid = ad.mycid;
```

Q3) Combine customers, accounts, and address (include customers missing in either)

```
SELECT c.cid, c.cname, c.phone, a.accno, a.bal, ad.city, ad.state  
FROM customers c  
LEFT JOIN accounts a ON c.cid = a.mycid  
LEFT JOIN address ad ON c.cid = ad.mycid;
```



6.7. Right Outer Join

- Returns all records from the right table and matching records from the left table.
- If there's no match, left table columns show NULL.

Syntax

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.common_col = table2.common_col;
```

Examples

Q1) Customers with accounts (include accounts without customers)

```
SELECT c.cid, c.cname, a.accno, a.bal
FROM customers c
RIGHT JOIN accounts a ON c.cid = a.mycid;
```

Q2) Customers with address (include addresses not linked to any customer)

```
SELECT c.cid, c.cname, ad.city, ad.state
FROM customers c
RIGHT JOIN address ad ON c.cid = ad.mycid;
```

Q3) Combine customers, accounts, and address (include orphan accounts or addresses)

```
SELECT c.cid, c.cname, c.phone, a.accno, a.bal, ad.city, ad.state
FROM customers c
RIGHT JOIN accounts a ON c.cid = a.mycid
RIGHT JOIN address ad ON c.cid = ad.mycid;
```



6.8. Full Outer Join

- Combines LEFT and RIGHT JOIN results — returns all matching and non-matching rows from both tables.
- MySQL doesn't support FULL OUTER JOIN directly — use UNION.

Example (MySQL)

```
SELECT * FROM customers c
LEFT JOIN accounts a ON c.cid = a.mycid
UNION
SELECT * FROM customers c
RIGHT JOIN accounts a ON c.cid = a.mycid;
```

6.9. JOINS with Extra Conditions

- You can filter joined results using additional conditions

Examples

Q1) Customers with balance >= 15000

```
SELECT * FROM customers c
INNER JOIN accounts a
ON c.cid = a.mycid WHERE bal >= 15000;
```

Q2) Customers in Blore

```
SELECT * FROM customers c
INNER JOIN address ad
ON c.cid = ad.mycid WHERE city = 'Blore';
```

Q3) Customers in Blore or Pune with balance >=15000

```
SELECT * FROM customers c
INNER JOIN accounts a ON c.cid = a.mycid AND bal >= 15000
INNER JOIN address ad ON c.cid = ad.mycid AND city IN ('Blore','Pune');
```



6.10. Self Join

- A table joined with itself to compare rows within the same table.

Example

```
CREATE TABLE myemployees (  
    empId INT(3),  
    empName CHAR(15),  
    mgrId INT(3)  
);
```

```
INSERT INTO myemployees VALUES  
(101,'sri',103),  
(102,'vas',103),  
(103,'sd',NULL),  
(104,'ds',101),  
(105,'aaa',101),  
(106,'bbb',102);
```

Q1) Display employee and manager names

```
SELECT emp.empName AS "Employee", mgr.empName AS "Manager"  
FROM myemployees emp  
INNER JOIN myemployees mgr  
ON emp.mgrId = mgr.empId;
```

Result:

Employee	Manager
sri	sd
vas	sd
ds	sri
aaa	sri
bbb	vas



6.11. Cross Join

- Returns the Cartesian product — every record from the first table combined with every record from the second.

Syntax:

```
SELECT columns  
FROM table1  
CROSS JOIN table2;
```

Examples:

```
SELECT cid, cname, accno, bal  
FROM customers c, accounts a;
```

```
SELECT cid, cname, accno, bal  
FROM customers c  
CROSS JOIN accounts a;
```

Note:

- ✓ If customers has 9 rows and accounts has 8 rows
- ✓ → total = $9 \times 8 = 72$ combinations.



7. Subqueries

- Learn how to use Subqueries (Nested Queries) to retrieve data dynamically from other queries,
- and how to use Derived Tables to create temporary, in-query result sets.

7.1. What is a Subquery?

7.2. Rules of Subqueries

7.3. Types of Subqueries

7.4. Practice Lab - Subqueries

7.5. Creating New Tables from Queries



7.1. What is a Subquery?

- Subquery is a query nested inside another SQL query.
- Inner query (subquery) executes first, and its result is passed to the outer query (main query).
- Subqueries can be placed inside:
SELECT
FROM
WHERE
HAVING clauses.

Terminology:

- Subquery → Inner query that runs first
- Main Query → Outer query that uses subquery result

Execution Order:

- Inner query → executes first → result passed to main query

7.2. Rules of Subqueries

- Subquery must be enclosed in parentheses ().
- It can return single or multiple values.
- When the subquery returns one value, use = operator.
- When the subquery returns multiple values, use IN, ANY, or ALL.

7.3. Types of Subqueries

- a) Single-Row Subquery
- b) Multi-Row Subquery

Note : Advanced types like Correlated and Derived Tables will be covered in Module 2: Advanced SQL

a) Single-Row Subquery

- Subquery that returns only one row of result.
- Used With Operators: =, <, >, <=, >=, <>

Example:

```
SELECT sname, bal
FROM mystudents
WHERE bal > (
    SELECT AVG(bal)
    FROM mystudents
);
```

Compares each student's balance with the average balance (one value).

b) Multi-Row Subquery

- Subquery that returns multiple rows.
- Used With Operators: **IN**, **ANY**, **ALL**

Example:

```
SELECT sname, city
FROM mystudents
WHERE city IN (
    SELECT city
    FROM mystudents
    WHERE course = 'ML'
);
```

Returns students who are in the same cities as ML students.



7.4. Practice Lab - Subqueries

Step 1: Create Tables

Use the same 3 tables from previous lab: customers, accounts, and address

Q1) Display the balance of the customer whose phone is 333

Without Subquery

```
SELECT cid FROM customers WHERE phone = 333;    # Inner Query
SELECT bal FROM accounts WHERE mycid = 103;     # Outer Query
```

With Subquery

```
SELECT bal
FROM accounts
WHERE mycid = (SELECT cid FROM customers WHERE phone = 333);
```

Q2) Display AccNo and Bal of customers who are staying in Blore

Without Subquery

```
SELECT mycid FROM address WHERE city = 'Blore';
SELECT accno, bal FROM accounts WHERE mycid IN (101, 102, 103);
```

With Subquery

```
SELECT accno, bal
FROM accounts
WHERE mycid IN (SELECT mycid FROM address WHERE city = 'Blore');
```

Q3) Display the City of customer whose email is 'ccc@jlc'

```
SELECT city
FROM address
WHERE mycid = (SELECT cid FROM customers WHERE email = 'ccc@jlc');
```



Q4) Display cname, email, and phone of the customer whose accno = 12345

```
SELECT cname, email, phone
FROM customers
WHERE cid = (SELECT mycid FROM accounts WHERE accno = 12345);
```

Q5) Display customers who are not maintaining minimum balance (< 20000)

```
SELECT *
FROM customers
WHERE cid IN (SELECT mycid FROM accounts WHERE bal < 20000);
```

Q6) Display cname, email, street, city of customers not maintaining Min Balance (< 20000)

```
SELECT cname, email, street, city
FROM customers c
INNER JOIN address a
ON c.cid = a.mycid
WHERE c.cid IN (SELECT mycid FROM accounts WHERE bal < 20000);
```

or (alternative using LEFT JOIN)

```
SELECT cname, email, street, city
FROM customers c
LEFT JOIN address a
ON c.cid = a.mycid
WHERE c.cid IN (SELECT mycid FROM accounts WHERE bal < 20000);
```

Subquery + join combination.



7.5. Creating New Tables from Queries

Q7) Create table with all columns and data from customers

```
CREATE TABLE mycust1 AS  
SELECT * FROM customers;
```

Q8) Create table with selected columns and data

```
CREATE TABLE mycust2 AS  
SELECT cid, cname, phone  
FROM customers;
```

Q9) Create table with structure only (no data)

```
CREATE TABLE mycust3 AS  
SELECT * FROM customers WHERE 1 = 2;
```

Q10) Create table with selected columns and no data

```
CREATE TABLE mycust4 AS  
SELECT cid, cname, phone  
FROM customers  
WHERE 1 = 2;
```

Q11) Create table with data filtered by condition

```
CREATE TABLE myadd1 AS  
SELECT * FROM address  
WHERE city = 'Blore';
```

Q12) Create table from customers, accounts, and address (joined)

```
CREATE TABLE cust_full_info AS  
SELECT c.cid, c.cname, c.email, a.accno, a.bal, ad.city, ad.state  
FROM customers c  
INNER JOIN accounts a ON c.cid = a.mycid  
INNER JOIN address ad ON c.cid = ad.mycid;
```



8. Set Operations

- Learn how to combine results from multiple SELECT queries using SQL Set Operators.

8.1. What are Set Operations?

8.2. Rules for Using Set Operators

8.3. Types of Set Operations

8.4. Practice Lab – Set Operations

8.5. ORDER BY with Set Operations



8.1. What are Set Operations?

- Allow you to combine two or more SELECT query results into a single result set.
- They work similar to mathematical set operations
 - Union, Intersection, and Difference.

8.2. Rules for Using Set Operators

- Each query must have the same number of columns.
- Columns must have compatible data types.
- Column order must match in each query.
- ORDER BY can be applied only at the end of the final combined query.

8.3. Types of Set Operations

- Types of Set Operations
 - ✓ UNION
 - ✓ UNION ALL
 - ✓ INTERSECT
 - ✓ EXCEPT / MINUS

8.4. Practice Lab – Set Operations

Step 1: Create Tables

```
CREATE TABLE students1 (  
    sid INT,  
    sname CHAR(15),  
    course CHAR(15)  
);
```

```
CREATE TABLE students2 (  
    sid INT,  
    sname CHAR(15),  
    course CHAR(15)  
);
```



Step 2: Insert Sample Records

```
INSERT INTO students1 VALUES  
(101, 'Srinivas', 'Java'),  
(102, 'Vas', 'Python'),  
(103, 'Sd', 'ML'),  
(104, 'Ds', 'SQL');
```

```
INSERT INTO students2 VALUES  
(103, 'Sd', 'ML'),  
(104, 'Ds', 'SQL'),  
(105, 'Hello', 'Python'),  
(106, 'Hai', 'AI');
```

A) UNION

- Combines the results of two or more SELECT queries into a single result set.
- Removes duplicate rows automatically — only unique records are included in the final output.

Ex:

- ✓ Combine customer lists from two different branches without duplicates.

Syntax:

```
SELECT column_list FROM table1  
UNION  
SELECT column_list FROM table2;
```

Example:

```
SELECT sname, course FROM students1  
UNION  
SELECT sname, course FROM students2;
```



B) UNION ALL

- Combines all results from multiple SELECT queries, including duplicates.
- Keeps all rows — does not check for or remove duplicates.

Ex:

- ✓ UNION ALL is faster because it skips duplicate checks, ideal for combining logs or time-series data.

Syntax:

```
SELECT column_list FROM table1  
INTERSECT  
SELECT column_list FROM table2;
```

Example:

```
SELECT sname, course FROM students1  
INTERSECT  
SELECT sname, course FROM students2;
```

C) INTERSECT

- Returns only those records that appear in both SELECT queries —common elements.
- Removes duplicates automatically — shows each common record only once.

Ex:

- ✓ Find common users, transactions, or employees existing in two departments or datasets.

Syntax:

```
SELECT column_list FROM table1  
UNION  
SELECT column_list FROM table2;
```

Example:

```
SELECT sname, course FROM students1  
UNION  
SELECT sname, course FROM students2;
```

✓ **Note:** MySQL doesn't support INTERSECT directly.

✓ Use an INNER JOIN alternative:

Example:

```
SELECT a.sname, a.course
FROM students_a a
INNER JOIN students_b b
ON a.sname = b.sname AND a.course = b.course;
```

D) EXCEPT / MINUS

- Returns records from the first query that do not exist in the second query.
- Removes duplicates from the final result — each unique record from the first dataset appears only once if it's not in the second.

Ex:

✓ Find customers who registered last year but have not made any purchases this year.

Syntax:

```
SELECT column_list FROM table1
EXCEPT
SELECT column_list FROM table2;
```

Example:

```
SELECT sname, course FROM students1
EXCEPT
SELECT sname, course FROM students2;
```

✓ **Note:** MySQL doesn't support EXCEPT or MINUS directly.

✓ You can simulate it using a LEFT JOIN and WHERE IS NULL:

Example:

```
SELECT a.sname, a.course
FROM students_a a
LEFT JOIN students_b b
ON a.sname = b.sname AND a.course = b.course
WHERE b.sname IS NULL;
```



8.5. ORDER BY with Set Operations

- To sort combined results, apply ORDER BY only once at the end.

Example:

```
SELECT sname, course FROM students_a  
UNION  
SELECT sname, course FROM students_b  
ORDER BY sname ASC;
```



9. Views

9.1. Introduction to Views

9.2. Practice Lab – Views

9.3. Types of Views (by updatability)



9.1. Introduction to Views

- VIEW is a saved SQL query that behaves like a virtual (logical) table.
- It stores query logic (schema), not the data itself; data is read from the underlying table(s).
- Great for column/row level restriction, simplifying complex queries.

Syntax:

Create view definition

```
CREATE VIEW view_name AS  
SELECT column_list  
FROM base_table(s)  
[WHERE ...];
```

Modify an existing view definition

```
CREATE OR REPLACE VIEW view_name AS  
SELECT ...;
```

Remove a view

```
DROP VIEW view_name;
```

9.2. Practice Lab – Views

Step 1: Base Table (simple, single source for easy updatable views)

```
CREATE TABLE customers (  
  cid INT(3) PRIMARY KEY,  
  cname CHAR(15) NOT NULL,  
  email CHAR(15) NOT NULL UNIQUE,  
  phone INT(10) NOT NULL UNIQUE,  
  city CHAR(15) NOT NULL,  
  status CHAR(15) NOT NULL,  
  accno INT(6) NOT NULL UNIQUE,  
  atype CHAR(2) NOT NULL,  
  branch CHAR(10) NOT NULL,  
  bal DOUBLE  
);
```



Step 2: Insert Sample Records

```
INSERT INTO customers VALUES
(101,'sri','sri@myjlc',123,'Blore','Active',5001,'SA','B01',25000),
(102,'vas','vas@myjlc',321,'Blore','Active',5002,'SA','B01',25000),
(103,'sd','sd@myjlc',234,'Hyd','Active',5003,'SA','B01',25000),
(104,'ds','ds@myjlc',345,'Hyd','Active',5004,'SA','B01',25000),
(105,'aa','aa@myjlc',111,'Delhi','Active',5005,'SA','B01',25000),
(106,'bb','bb@myjlc',222,'Delhi','Active',5006,'SA','B01',25000),
(107,'cc','cc@myjlc',333,'Blore','Active',5007,'SA','B01',25000),
(108,'dd','dd@myjlc',444,'Blore','Active',5008,'SA','B01',25000),
(109,'ee','ee@myjlc',555,'Hyd','Active',5009,'SA','B01',25000),
(110,'ff','ff@myjlc',666,'Hyd','Active',5010,'SA','B01',25000),
(111,'gg','gg@myjlc',777,'Delhi','Active',5011,'SA','B01',25000),
(112,'hh','hh@myjlc',888,'Delhi','Active',5012,'SA','B01',25000);
```

3) Teller View (limited columns)

Create View:

```
CREATE VIEW teller_view AS
SELECT cid, cname, city, accno, atype, bal
FROM customers;
```

Fetch data from View:

```
SELECT * FROM teller_view;           # Runs Successfully
SELECT email FROM teller_view;       # Error (not in the view)
```

4) City-wise Teller Views

Create View:

```
CREATE VIEW blore_teller_view AS
SELECT cid, cname, city, accno, atype, bal
FROM customers
WHERE city = 'Blore';
```

```
CREATE VIEW hyd_teller_view AS
SELECT cid, cname, city, accno, atype, bal
FROM customers
WHERE city = 'Hyd';
```

```
CREATE VIEW delhi_teller_view AS
SELECT cid, cname, city, accno, atype, bal
FROM customers
WHERE city = 'Delhi';
```

Fetch data from View:

```
SELECT * FROM blore_teller_view;
SELECT * FROM hyd_teller_view;
SELECT * FROM bdelhi_teller_view;
```



9.3. Types of Views (by updatability)

- Two Types of Views
 - ✓ A) Read-Only Views
 - ✓ B) Updatable Views

A) Read-Only Views

- Also Called as Static Views
- You can SELECT but cannot INSERT/UPDATE/DELETE through the view.
- **Common reasons a view becomes read-only:**
 - ✓ It uses GROUP BY, DISTINCT, HAVING, UNION, aggregates.
 - ✓ It selects from multiple tables (joins) without special support.
 - ✓ It omits key columns needed to uniquely identify rows.
 - ✓ It has subqueries in the SELECT list

B) Updatable Views

- Also Called as Dynamic Views
- You can INSERT/UPDATE/DELETE through the view and changes reflect in the base table.
- **Typical requirements (single-table views):**
 - ✓ Based on one table (no aggregates, no GROUP BY, no UNION).
 - ✓ Includes the primary key (or enough columns to uniquely identify rows).
 - ✓ No computed/derived columns in the SELECT list.
 - ✓ No DISTINCT.



10. SQL Built-In Functions

10.1. Arithmetic Functions

10.2. String Functions

10.3. Date & Time Functions

10.4. Conversion Functions

10.5. Aggregate Functions



10.1. Arithmetic Functions

- Used for mathematical calculations such as powers, square roots, rounding, etc.
- **Common Functions**

Function	Purpose
SQRT(x)	Returns the square root of x
POW(x, y)	Raises x to the power of y
MOD(x, y)	Returns remainder of $x \div y$
ABS(x)	Returns absolute value of x
CEIL(x)	Rounds x up to next integer
FLOOR(x)	Rounds x down to previous integer
ROUND(x)	Rounds x to nearest integer

Examples

```
SELECT SQRT(25);           -- 5
SELECT POWER(25,2);        -- 625
SELECT POWER(25,0.5);      -- 5
SELECT MOD(10,3);          -- 1
SELECT ABS(-9);            -- 9
SELECT CEIL(5.2);          -- 6
SELECT FLOOR(5.9);         -- 5
SELECT ROUND(5.6);         -- 6
```



10.2. String Functions

- Used to manipulate or inspect text data.
- **Common Functions**

Function	Purpose
LENGTH(str)	Returns length of a string
TRIM(str)	Removes spaces from both sides
LTRIM()	Removes spaces from left side
RTRIM()	Removes spaces from right side
LPAD(str,n,pad)	Pads string to length n with pad characters
RPAD(str,n,pad)	Pads string to length n with pad characters
SUBSTR(str,pos,len)	Extracts substring
LOWER() / LCASE()	Converts to lowercase
UPPER() / UCASE()	Converts to uppercase
ASCII(char)	Returns ASCII value of a character
CHAR(code)	Returns character for an ASCII code
CONCAT(str1,str2,...)	Joins multiple strings
REPLACE(str,old,new)	Replaces substring

Examples

```
SELECT LENGTH('Srinivas');           -- 8
SELECT TRIM(' Sri nivas ');          -- 'Sri nivas'
SELECT RPAD('JLC',6,'*');            -- 'JLC***'
SELECT LPAD('JLC',5,'*');            -- '**JLC'
SELECT SUBSTR('Srinivas',3,4);        -- 'iniv'
SELECT LOWER('SriNiVas');            -- 'srinivas'
SELECT UPPER('SriNiVas');            -- 'SRINIVAS'
SELECT ASCII('A');                   -- 65
SELECT CHAR(97);                     -- 'a'
SELECT CONCAT('hello','guys');        -- 'helloguys'
SELECT REPLACE('helloguys','hello','JLC'); -- 'JLCguys'
```



10.3. Date & Time Functions

- Used to handle date/time values and extract parts of a date..
- **Common Functions**

Function	Purpose
<code>SYSDATE() / NOW()</code>	Current date & time
<code>DATE(expr)</code>	Extracts date part
<code>TIME(expr)</code>	Extracts time part
<code>DAY(expr)</code>	Returns day of month
<code>MONTH(expr)</code>	Returns month number
<code>YEAR(expr)</code>	Returns year

Examples

```
SELECT SYSDATE();           -- 2025-11-06 11:25:00
SELECT DATE(SYSDATE());     -- 2025-11-06
SELECT TIME(NOW());         -- 11:25:00
SELECT DAY(SYSDATE());      -- 6
SELECT MONTH(SYSDATE());    -- 11
SELECT YEAR(SYSDATE());     -- 2025
```

Formatting Dates

- Use MySQL's `DATE_FORMAT()` and `TIME_FORMAT()`

```
SELECT DATE_FORMAT(SYSDATE(), '%D %M %Y');    -- '6th November 2025'
SELECT DATE_FORMAT(SYSDATE(), '%d-%m-%Y');    -- '06-11-2025'
SELECT DATE_FORMAT(SYSDATE(), '%W %d-%m-%Y'); -- 'Thursday 06-11-2025'
SELECT DATE_FORMAT(SYSDATE(), '%d-%b-%Y');    -- '06-Nov-2025'
SELECT TIME_FORMAT(SYSDATE(), '%r');          -- 12-hour format
SELECT TIME_FORMAT(SYSDATE(), '%T');          -- 24-hour format
```



10.4. Conversion Functions

- Used to convert between data types and formats.
- **Common Functions**

Function	Purpose
STR_TO_DATE(str,format)	Converts a string into a DATE value based on format

Example:

```
CREATE TABLE hello ( id INT, hello DATE );  
INSERT INTO hello VALUES (101, STR_TO_DATE('10 August 2024','%d %M %Y'));  
SELECT * FROM hello;  
SELECT id, DATE_FORMAT(hello, '%d-%b-%Y') AS formatted_date FROM hello;
```

STR_TO_DATE() → converts string → DATE (for storage)
DATE_FORMAT() → converts DATE → string (for display)

10.5. Aggregate Functions

- Operate on multiple rows to produce a single result.
- **Common Functions**

Function	Purpose
COUNT()	Counts rows
SUM()	Adds numeric values
AVG()	Calculates average
MIN()	Finds minimum
MAX()	Finds maximum

Example:

```
SELECT  
    COUNT(*) AS total_students,  
    AVG(bal) AS avg_balance,  
    MAX(bal) AS highest_balance  
FROM mystudents;
```